

Data Visualisation - Vinci Reloaded 2026

Transforming Data Insights from Conceptual Design to Implementation

Magdalena A. Tkacz

Contents

The Visual Engineering Process	2
Part 1: Technical Setup & The Foundations of The Grammar of Graphics	2
1. Environmental Setup & Reproducible Documents	2
Global Document Configuration	2
2. Theoretical Framework: The Grammar of Graphics	3
Visual Pre-attentive Attributes	3
Understanding Data Mapping as Structural Layers	3
Part 2: Layered Visualization Mastery & Live ggplot2 Coding	4
1. Incremental Plot Building (The Layered Approach)	4
Step 1: Connecting Data to the Canvas	4
Step 2: Mapping Aesthetic Axis Dimensions	4
Step 3: Declaring Geometries (geom)	5
Step 4: Multi-dimensional Coding (Color Mapping)	6
Step 5: Professional Annotations and Labels	7
2. Advanced Object Manipulation, Trends, and Style Scales	8
Manual Color Assignment and Statistical Trends	8
Color Palette Extensions and Fixed Graphic Arguments	10
Dual Categorical Attribute Encoding (Shapes & Colors)	13
Structural Theme Alterations & Typography Settings	14
3. Alternative Geometries: Boxplots, Violins, and Hybrids	17
Comparing Geometric Output Formats	18
Hybrid Geometry Composition (Layer Stacking)	22
Intermediate Composite Plotting with Custom Jitter Overlays	23
4. One-Dimensional Layouts: Categorical Factor Sorting	24
Factor Reordering Techniques	27
5. Advanced Panel Layouts: Faceting and Marginal Displays	29

Split Axis Profiling via Matrix Facets	29
Appending Marginal Coordinates via ggExtra	34
Things I Found Useful & Interesting	37
DataViz Theory & Visual Communication Best Practices	37
The Grammar of Graphics & ggplot2 Documentation	37

The Visual Engineering Process

Before writing a single line of executable code, data visualization must be approached as a rigorous engineering discipline. The process is entirely **framework and tools agnostic**—implementation is simply the operationalization of a structured cognitive workflow:

- **Understand your data:** Continuously improve, iterate, and build a deep mental model of your underlying metrics.
- **Design with clarity:** Think deeply, decide deliberately, and choose the correct visual representation for your audience.
- **Encode with purpose:** Translate structural conceptual choices into clean, layered graphics.

Better decisions. Better charts. Stronger impact.

Part 1: Technical Setup & The Foundations of The Grammar of Graphics

1. Environmental Setup & Reproducible Documents

In scientific research, we utilize markdown environments to weave together text, statistical calculations, and code chunks into clean, production-ready outputs. While we use R for our practical sessions, the architectural choices remain identical across any engineering pipeline.

Global Document Configuration

We initialize our workshop environment by loading the necessary libraries, silencing scientific notation for clean axis formatting, and setting up a professional baseline theme.

```
knitr::opts_chunk$set(warning = FALSE, message = FALSE)
# Package installation if required:
# install.packages("ggplot2")
# install.packages("palmerpenguins")
# install.packages("forcats")
# install.packages("ggExtra")
# install.packages("RColorBrewer")

library(ggplot2)
library(palmerpenguins)
```

```
##
## Dołączanie pakietu: 'palmerpenguins'
```

```
## Następujące obiekty zostały zakryte z 'package:datasets':
##
##     penguins, penguins_raw

library(forcats)
library(ggExtra)
library(RColorBrewer)

# Disable scientific notation for clean axis rendering (e.g., 2000000 instead of 2e06)
options(scipen = 999)

# Set a crisp Black & White theme as our global structural baseline default
theme_set(theme_bw())
```

2. Theoretical Framework: The Grammar of Graphics

The **Grammar of Graphics** is a formalized system of rules that describes how statistical graphics are systematically constructed. Instead of treating a chart as a static image, this engineering methodology views visualization as a language where layouts are engineered by stacking independent, overlapping functional layers.

Visual Pre-attentive Attributes

Before the human brain consciously processes scales or numeric ranges, it automatically detects core visual attributes. By strategically mapping statistical variables to these pre-attentive cues, you instantly drive the reader's cognitive focus:

- **Form:** Length, width, sizing boundaries, and geometric shape configurations.
- **Color:** Hue (for category segregation) and intensity (for continuous numeric distributions).
- **Spatial Position:** 2D alignment across standard Cartesian coordinate systems.

Understanding Data Mapping as Structural Layers

Every chart is built from the ground up by stacking specific component layers:

1. **Data:** The raw data frame source containing your analytical metrics.
2. **Aesthetics (aes):** The structural link mapping a data variable to a visual property (e.g., X axis, Y axis, color, shape).
3. **Geometries (geom):** The concrete geometric shape used to render data points (points, bars, lines, boxes, violins).
4. **Facets:** Structural parameters that split the canvas into multiple sub-panels based on categorical variables.
5. **Statistics (stat):** Internal mathematical transformations automatically computed before plotting (e.g., bin counting for histograms, quantiles for boxplots).
6. **Coordinates:** The mapping space of the layout plane (Cartesian, polar, or inverted scales).
7. **Themes:** Non-data visual variables that control layout styling, text fonts, margins, gridlines, and canvas colors.

Part 2: Layered Visualization Mastery & Live ggplot2 Coding

1. Incremental Plot Building (The Layered Approach)

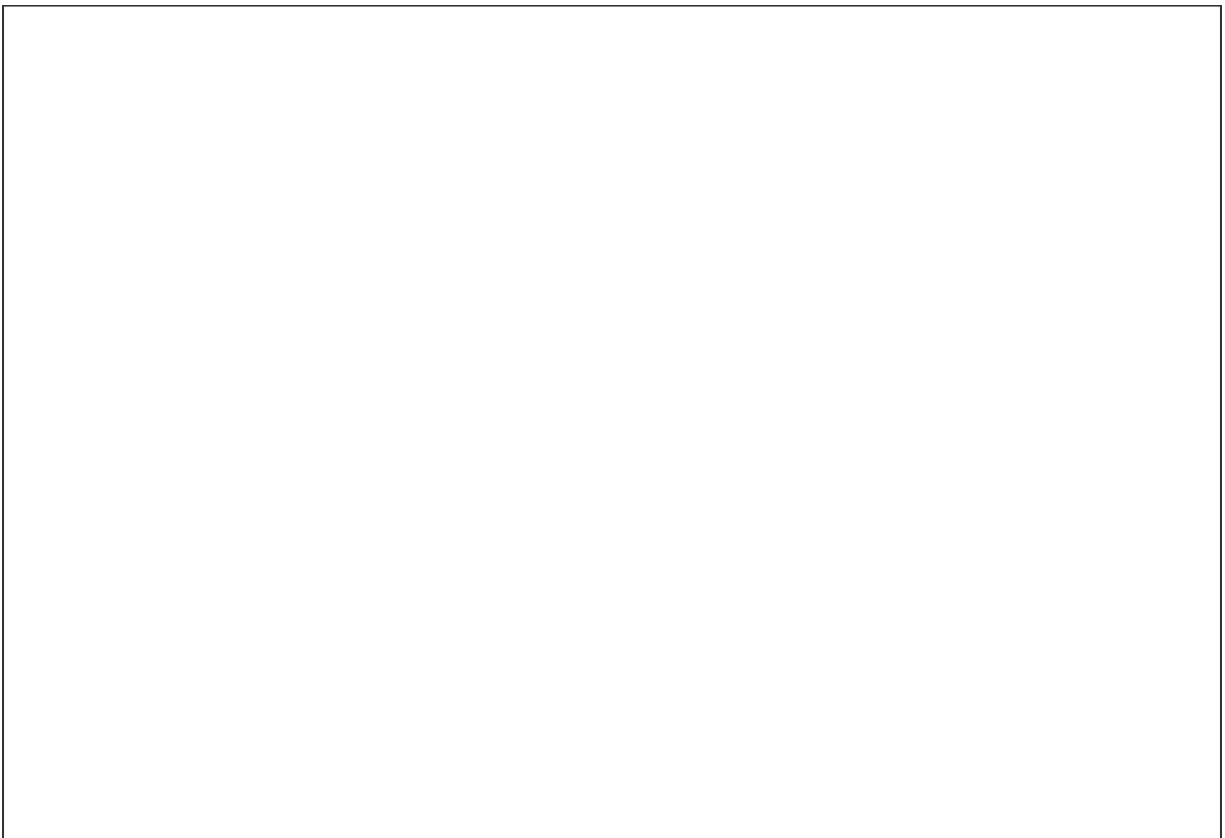
Charts in `ggplot2` are built iteratively using the base template: `ggplot(data = [dataset], mapping = aes([aesthetic_maps])) + [geom_functions]`.

Let's explore how a chart transforms step-by-step using the `palmerpenguins` dataset.

Step 1: Connecting Data to the Canvas

Initializing `ggplot()` establishes a data connection but renders an empty grey canvas because no spatial or aesthetic properties have been mapped.

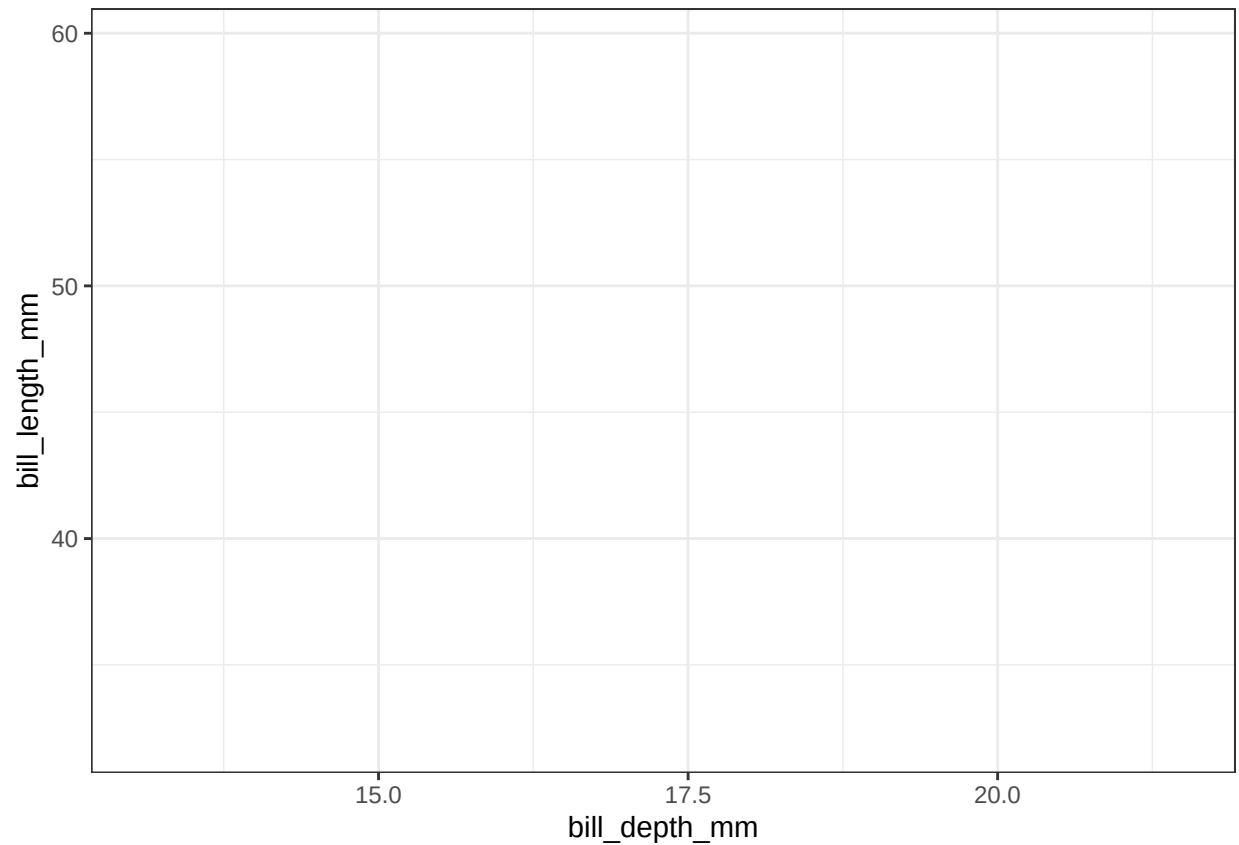
```
# Connects the dataset to the plotting engine  
ggplot(data = penguins)
```



Step 2: Mapping Aesthetic Axis Dimensions

Declaring variables inside `aes()` maps the numerical columns to the visual X and Y coordinate scales. The canvas now shows structured gridlines and numeric ranges but displays no data points.

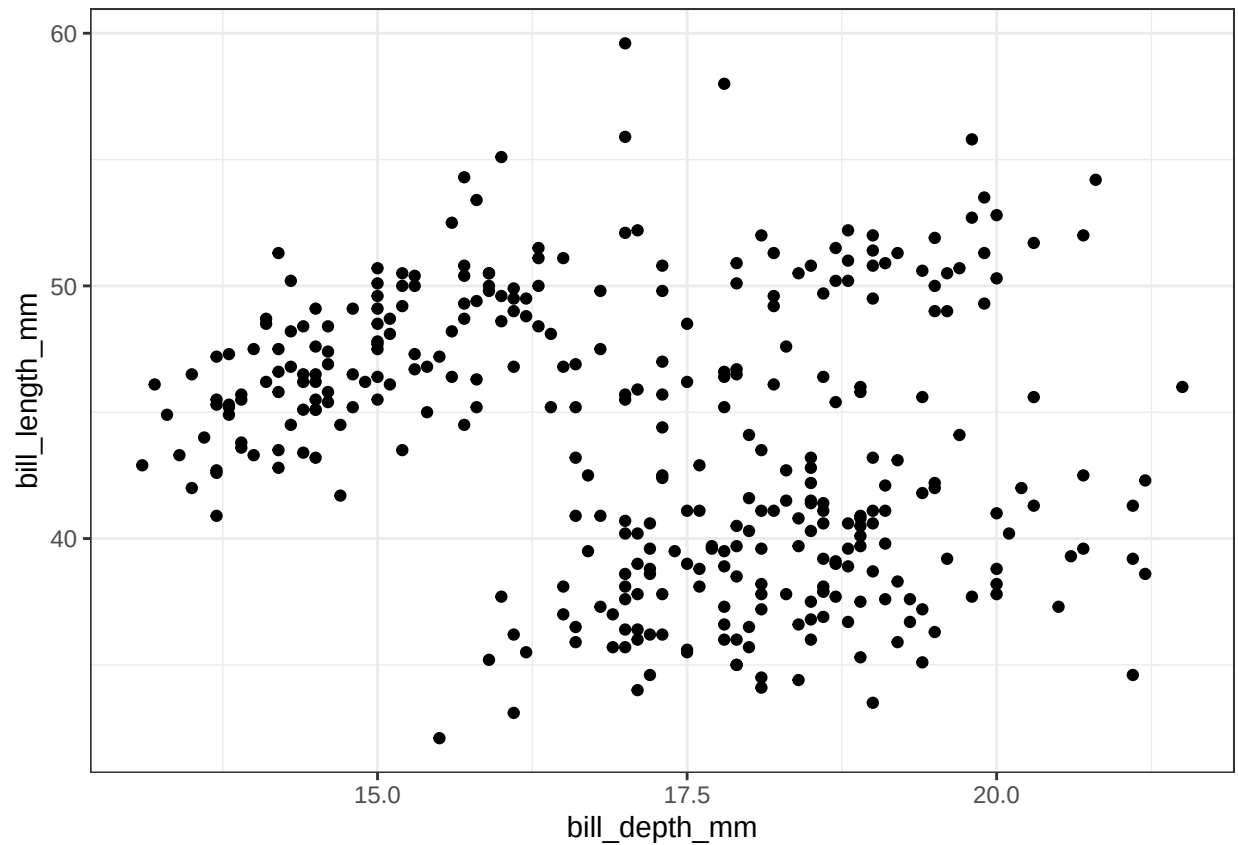
```
# Establishes structural scales for X and Y coordinates  
ggplot(data = penguins, aes(x = bill_depth_mm, y = bill_length_mm))
```



Step 3: Declaring Geometries (geom)

To project the data onto the canvas, we add a geometric layer. To inspect individual continuous data points, we append a scatter layer via `geom_point()`.

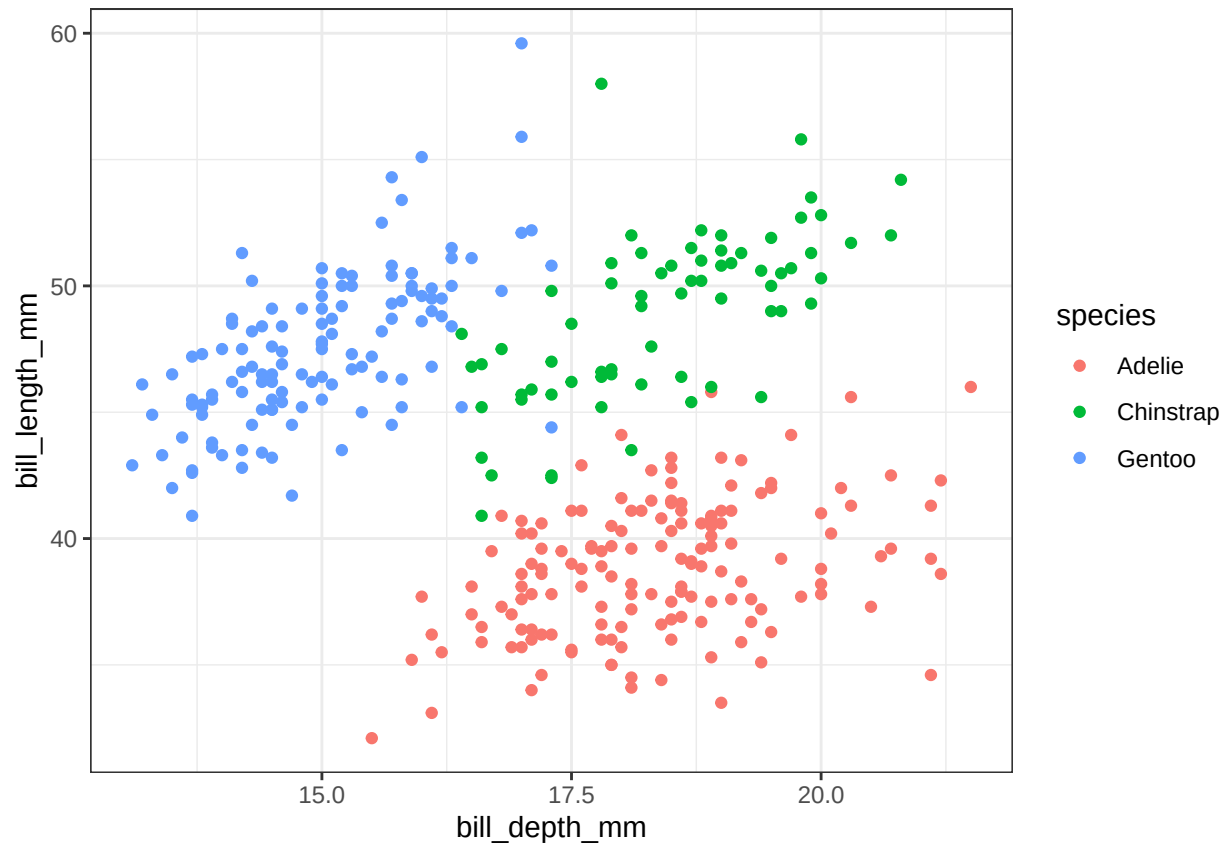
```
# Renders raw data rows as visual points on the canvas  
ggplot(data = penguins, aes(x = bill_depth_mm, y = bill_length_mm)) +  
  geom_point()
```



Step 4: Multi-dimensional Coding (Color Mapping)

To encode an additional categorical variable without cluttering the coordinates, we map the `species` factor to the pre-attentive property `color` inside the aesthetic function.

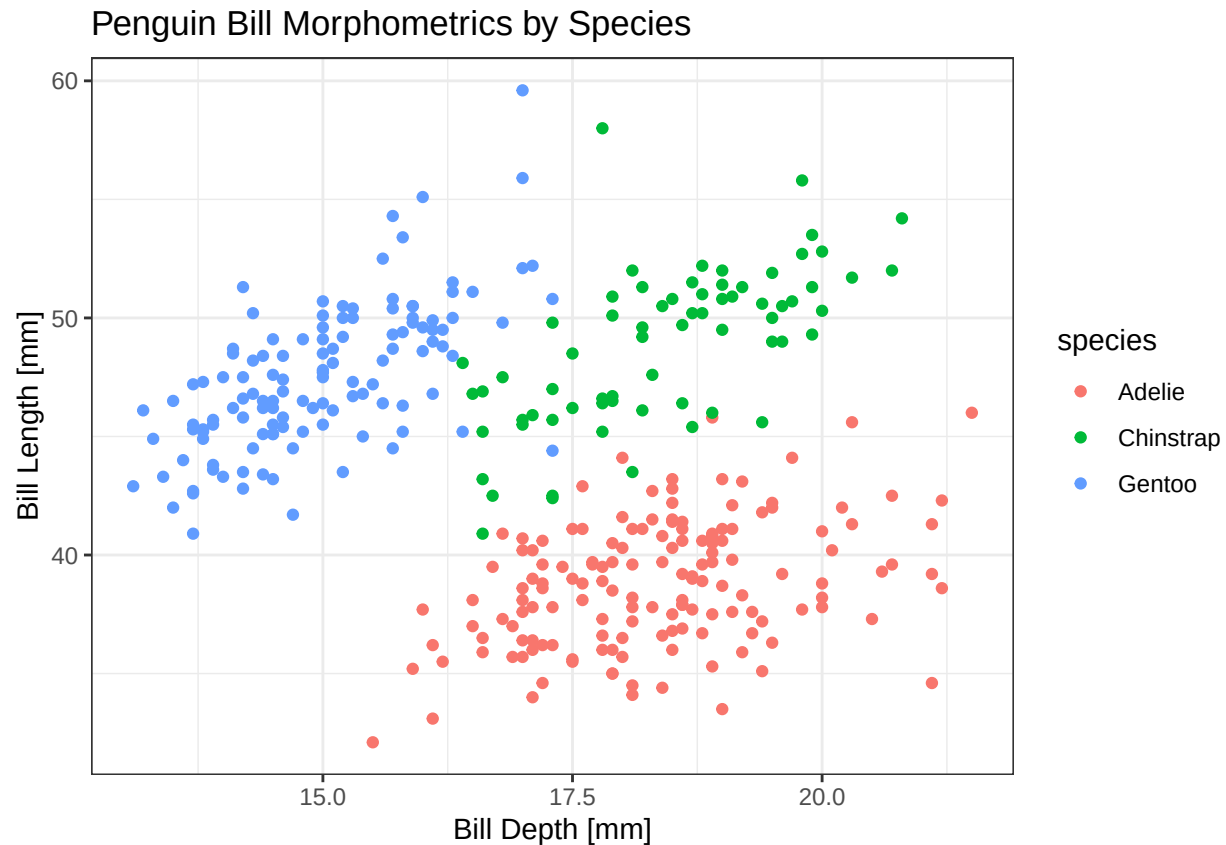
```
# Dynamically encodes biological species using colors  
ggplot(data = penguins,  
  aes(x = bill_depth_mm,  
    y = bill_length_mm,  
    col = species)) +  
geom_point()
```



Step 5: Professional Annotations and Labels

Every scientific visualization requires clear titles, clean labels, and explicit measurement units.

```
ggplot(data = penguins,  
  aes(x = bill_depth_mm,  
    y = bill_length_mm,  
    col = species)) +  
  geom_point() +  
  labs(title = "Penguin Bill Morphometrics by Species",  
    x = "Bill Depth [mm]",  
    y = "Bill Length [mm]")
```



2. Advanced Object Manipulation, Trends, and Style Scales

A distinct feature of `ggplot2` is that an entire visualization can be saved as a variable object, allowing you to manipulate and expand it later.

```
# Save our basic scatter configuration into a variable object
viz <- ggplot(data = penguins,
              aes(x = bill_depth_mm,
                  y = bill_length_mm,
                  col = species)) +
  geom_point() +
  labs(title = "Point Colors by Species",
       x = "Bill Depth [mm]",
       y = "Bill Length [mm]")
```

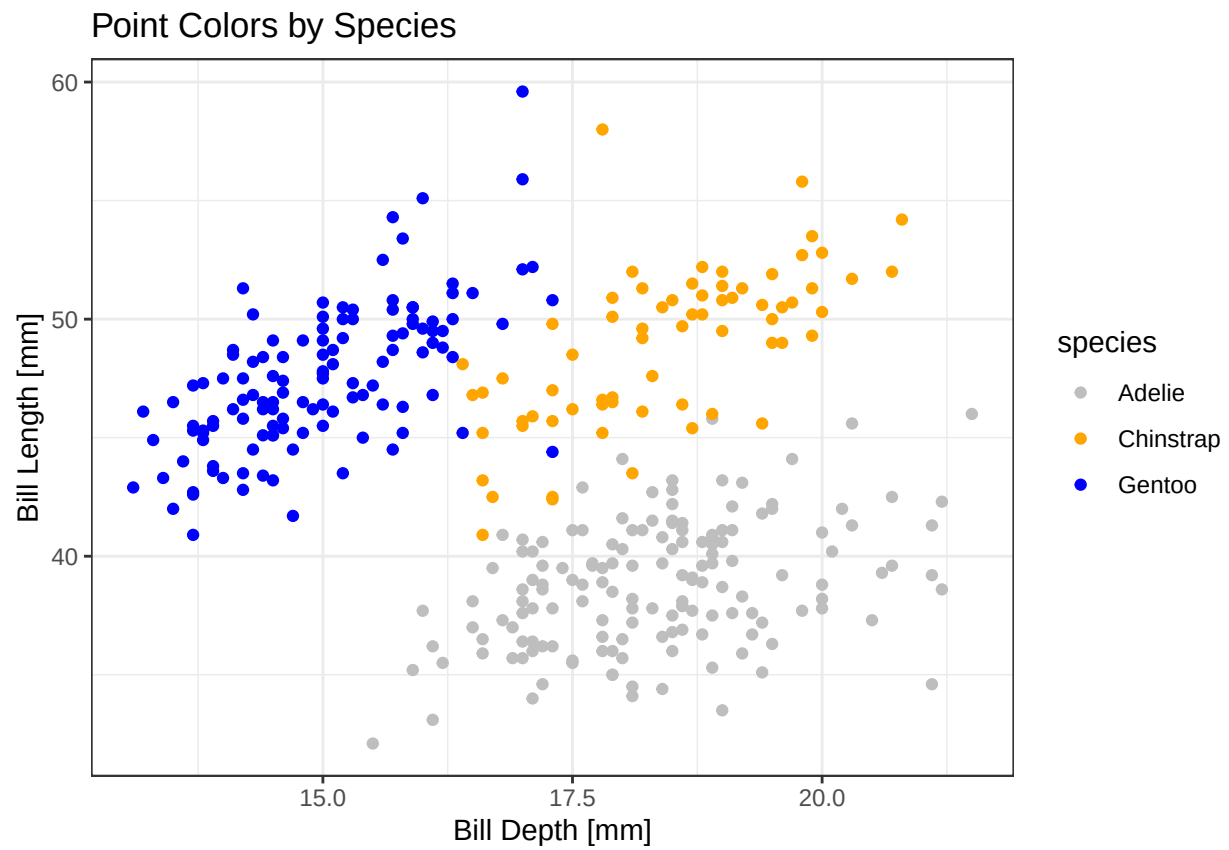
Manual Color Assignment and Statistical Trends

We can intercept default color schemes using manual scale modifications, and overlay statistical models like linear regressions with confidence ribbons using `geom_smooth()`.

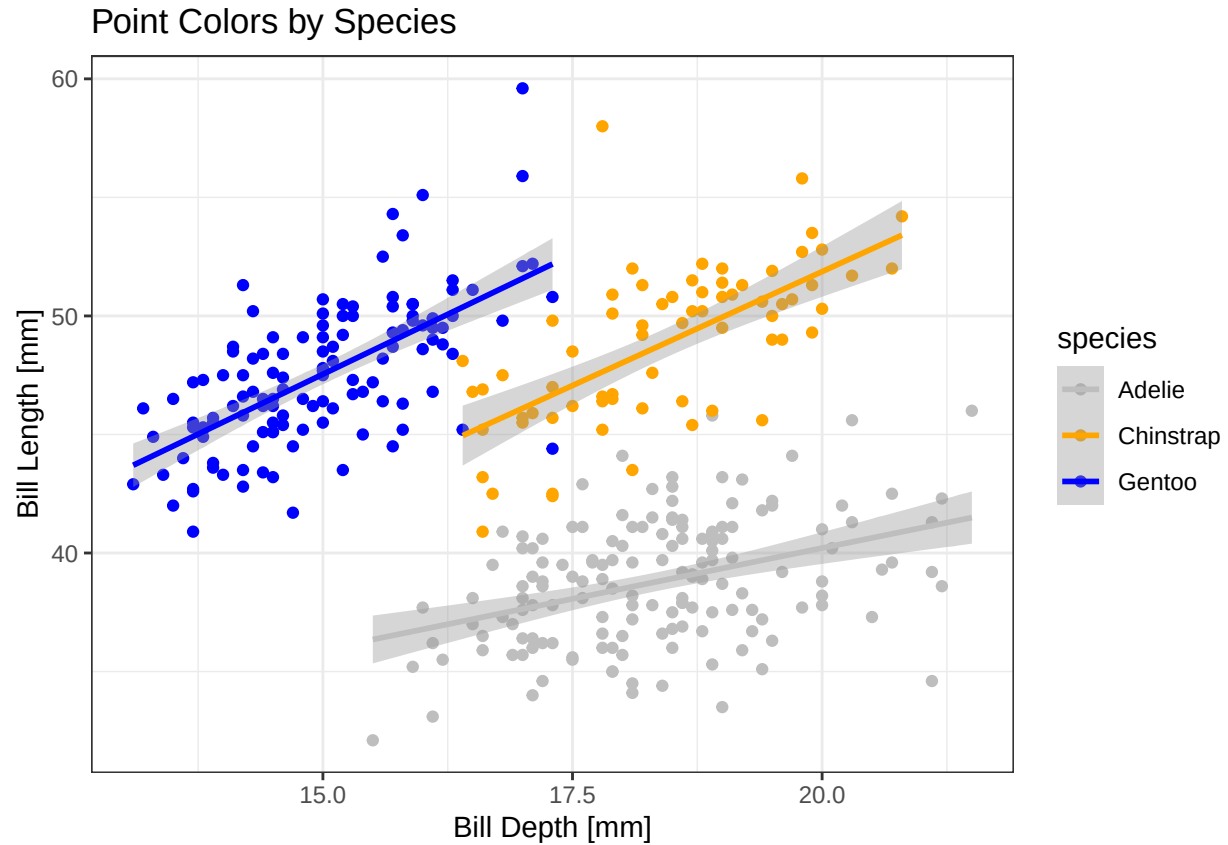
```
# Assign custom aesthetic values to the stored plot object
viz2 <- viz + scale_color_manual(values = c("gray", "orange", "blue"))
```



```
# Display the customized plot  
viz2
```



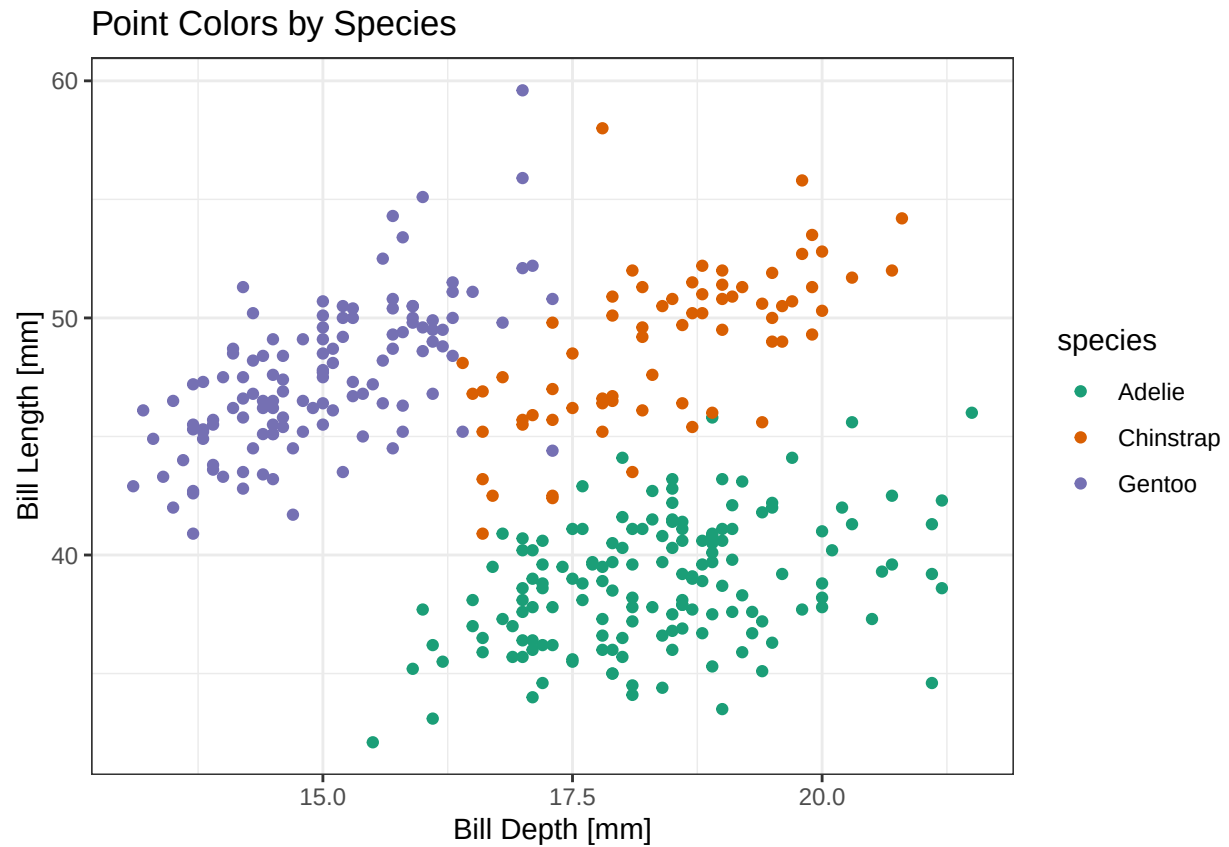
```
# Overlay a standard linear model regression trendline across mapped groups  
viz2 + geom_smooth(method = lm)
```



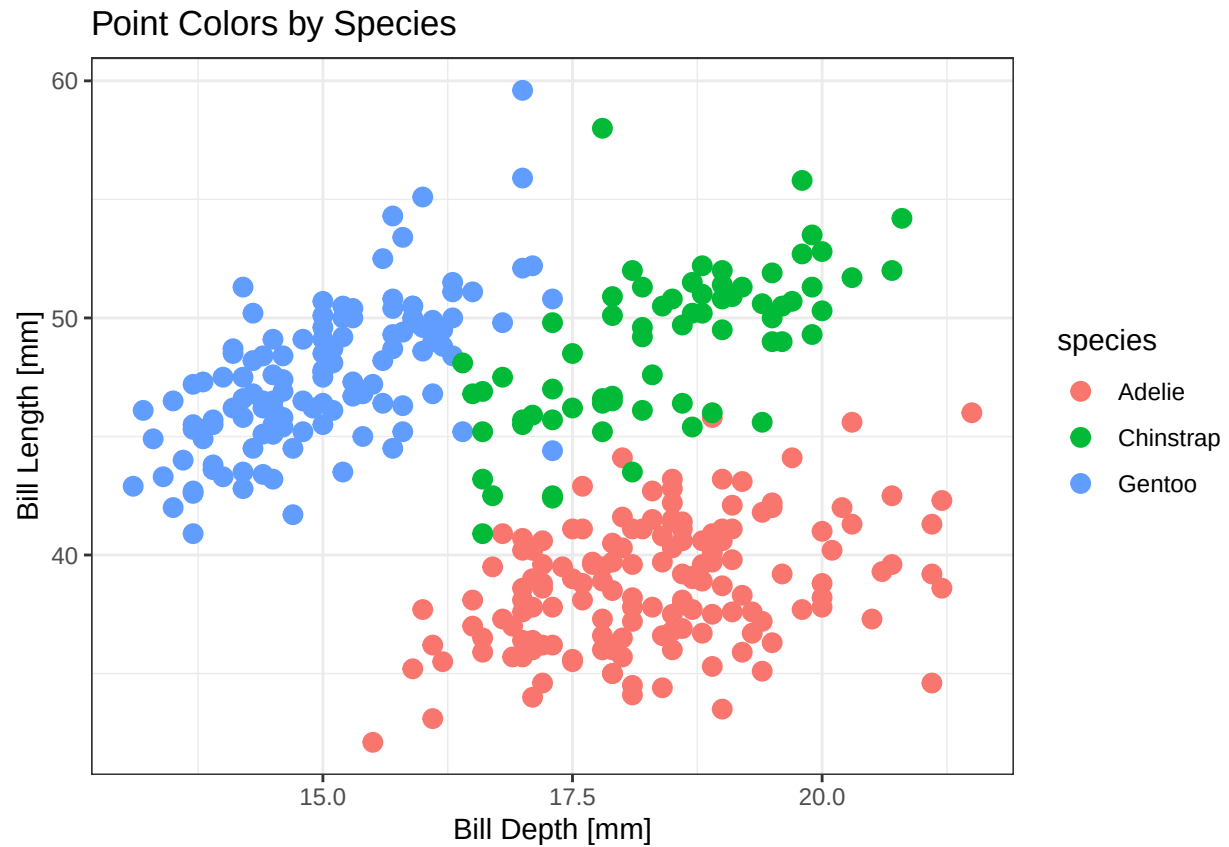
Color Palette Extensions and Fixed Graphic Arguments

Instead of choosing manual colors, you can use standardized scientific palettes like ColorBrewer. Alternatively, you can modify geometric sizing or shapes globally by passing constant arguments outside of the `aes()` function.

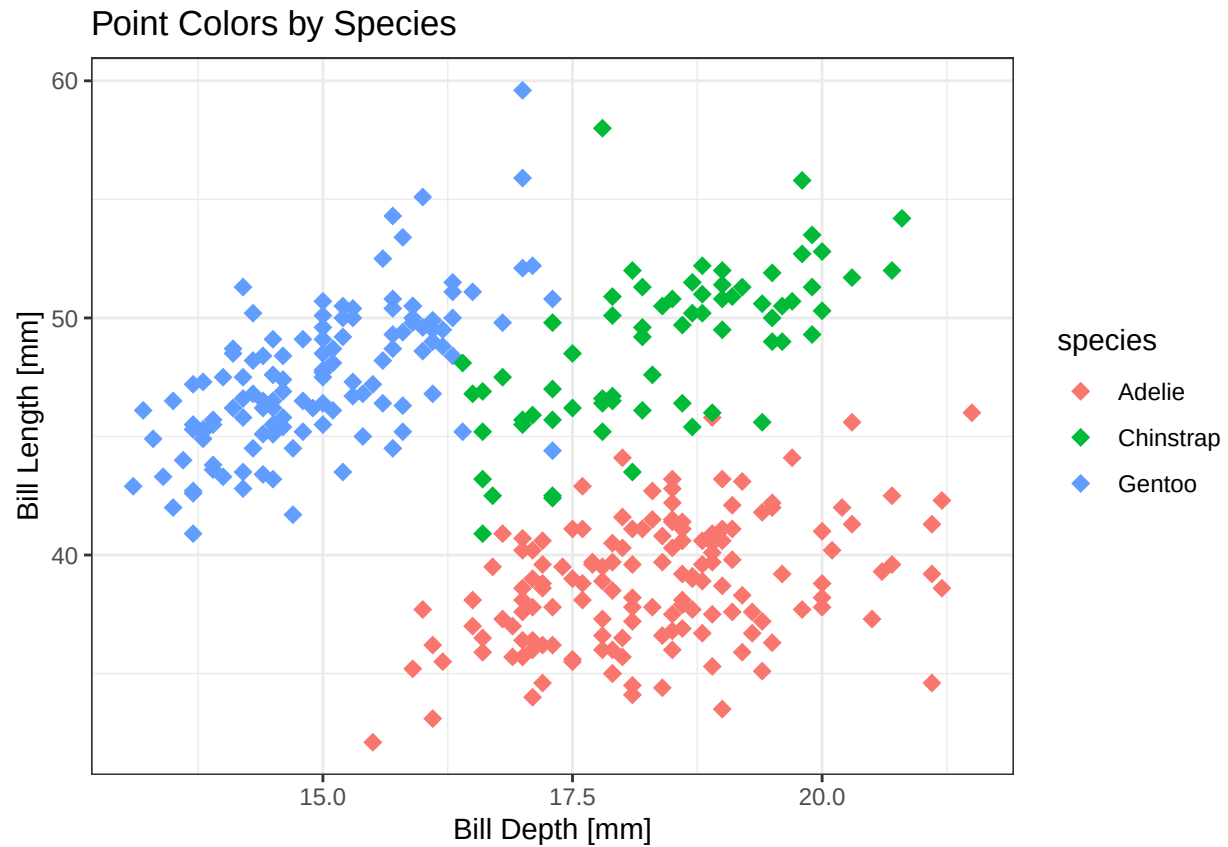
```
# Option A: Apply high-contrast ColorBrewer qualitative palettes  
viz + scale_color_brewer(palette = "Dark2")
```



```
# Option B: Increase visual point sizing globally (non-data mapping)  
viz + geom_point(size = 3)
```



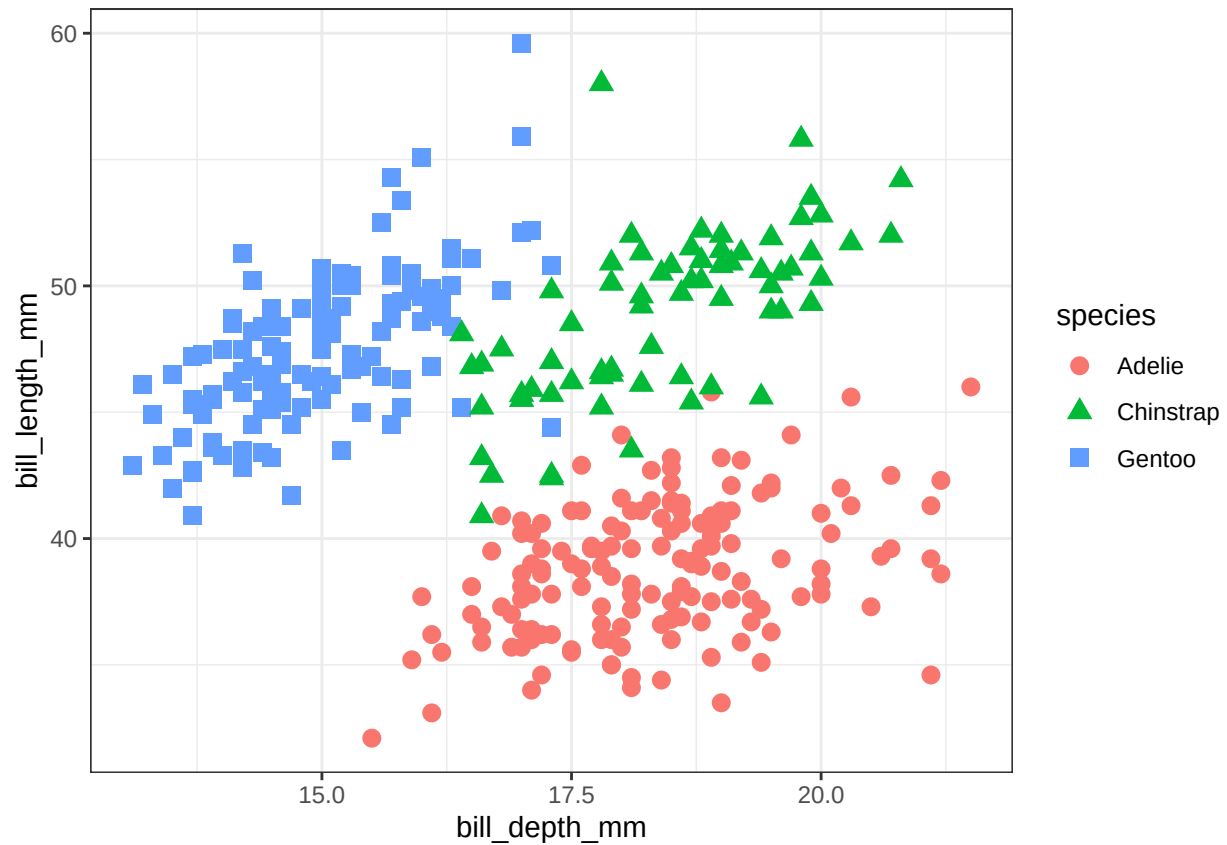
```
# Option C: Modify visual point shape using standard R shape integers  
viz + geom_point(size = 3, shape = 18)
```



Dual Categorical Attribute Encoding (Shapes & Colors)

To make graphics accessible for color-blind readers, encode categorical elements using both shape and color simultaneously.

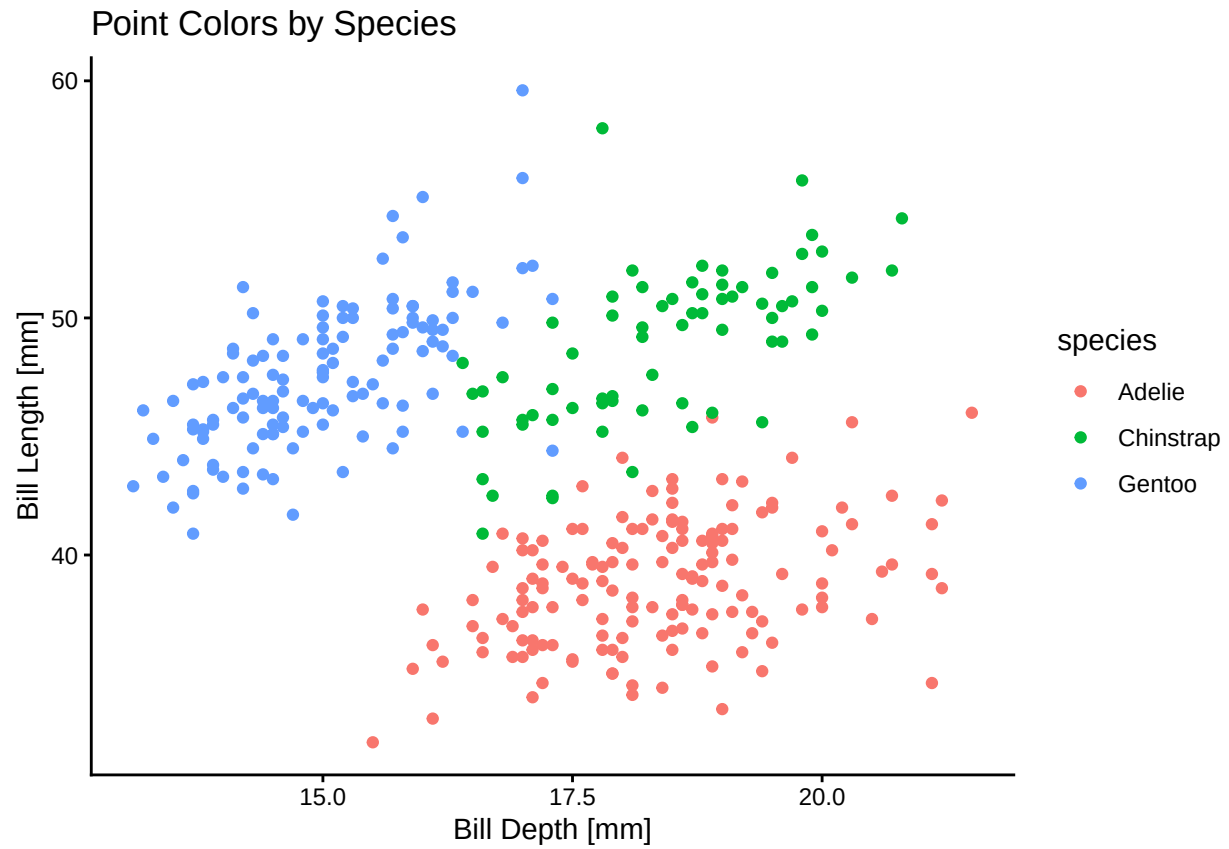
```
# Mapping species factors to both color and shape visual properties  
ggplot(data = penguins,  
  aes(x = bill_depth_mm,  
    y = bill_length_mm,  
    shape = species,  
    col = species)) +  
  geom_point(size = 3)
```



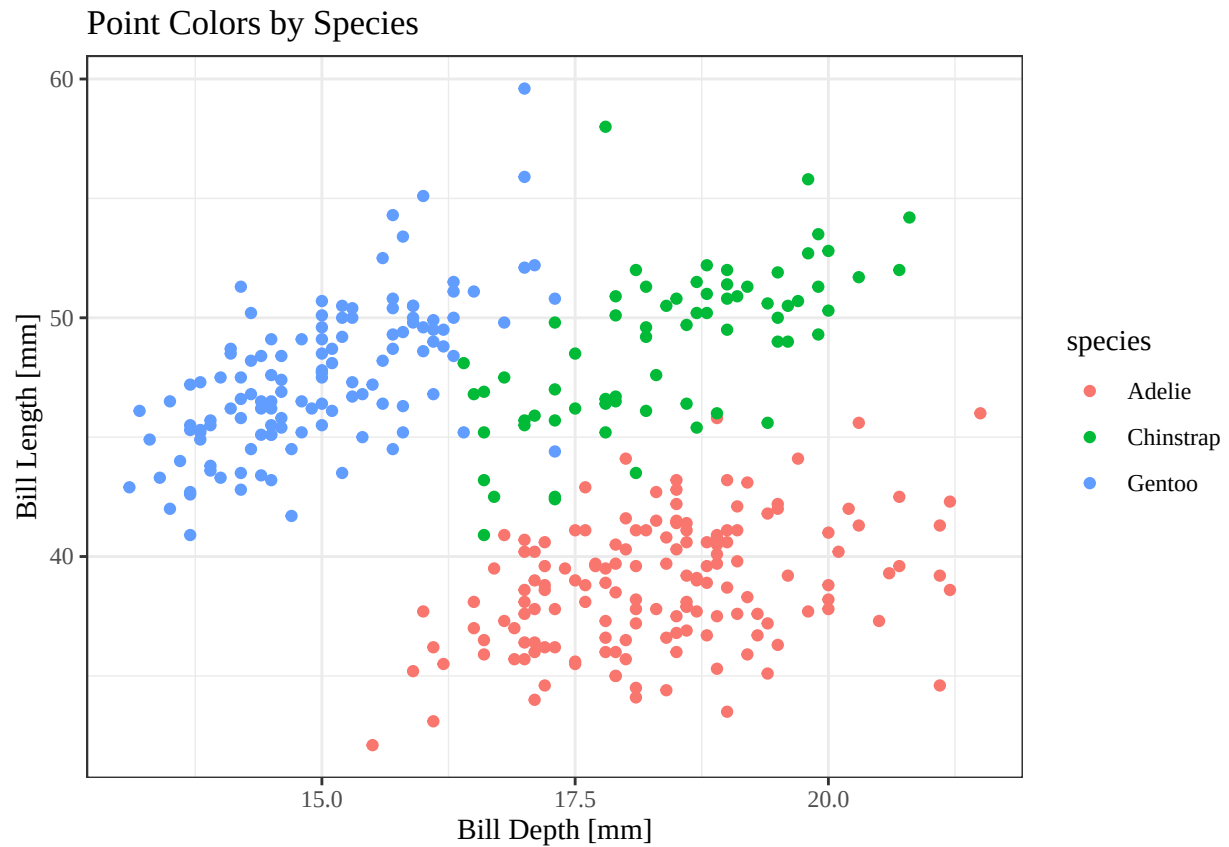
Structural Theme Alterations & Typography Settings

Themes control non-data design properties. You can easily switch layout aesthetics or inject custom typography parameters.

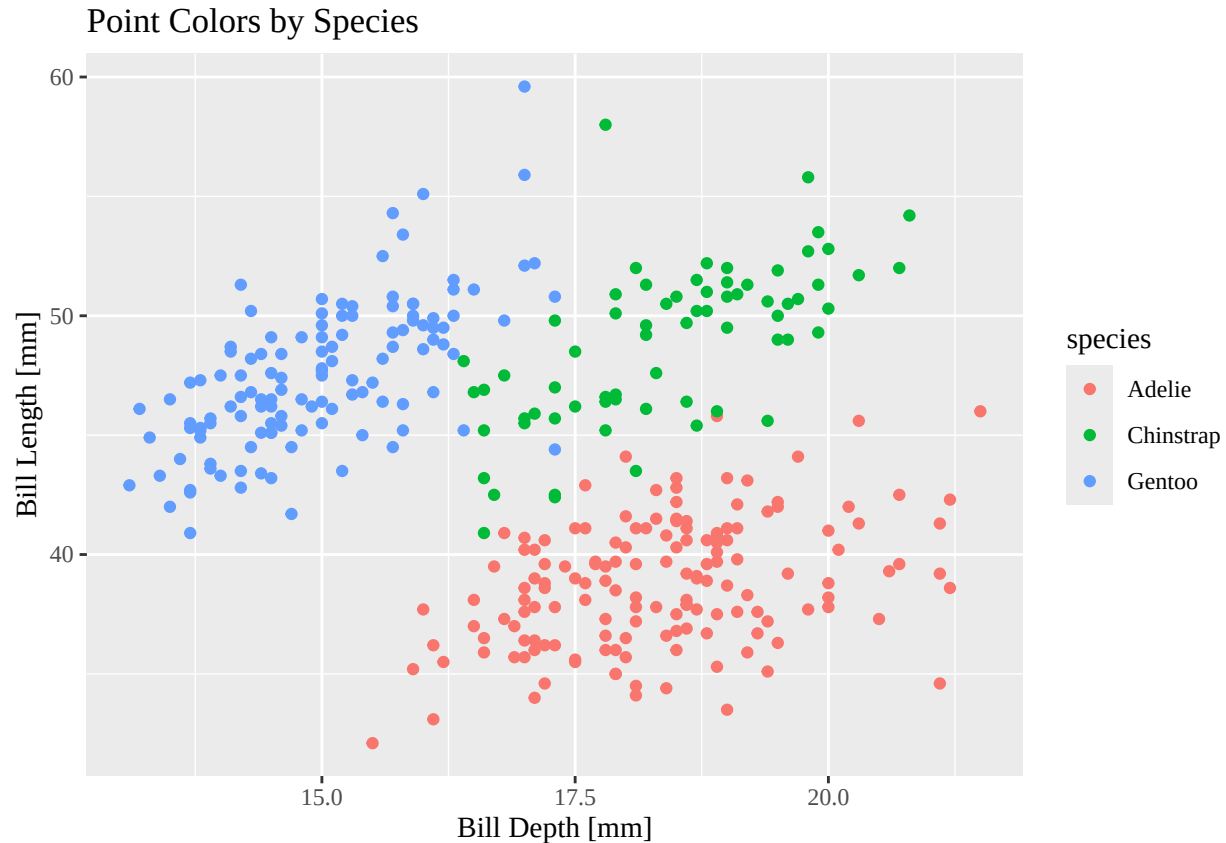
```
# Option A: Apply a minimalist classic theme layout  
viz + theme_classic()
```



```
# Option B: Force global typography modification using element_text parameters  
viz + theme(text = element_text(family = "serif"))
```



```
# Option C: Chain structural layout themes and individual overrides sequentially  
viz +  
  theme_gray() +  
  theme(text = element_text(family = "serif"))
```

3. Alternative Geometries: Boxplots, Violins, and Hybrids

To change how data distributions are represented, we only need to swap out the `geom_*` function layer. The underlying dataset and mapped scales remain completely identical.

Important Technical & Analytical Disclaimer:

The code examples below are designed to demonstrate **how** to technically swap geometry layers (`geom_*`) within the programming syntax. They do not necessarily represent correct or meaningful data visualization choices.

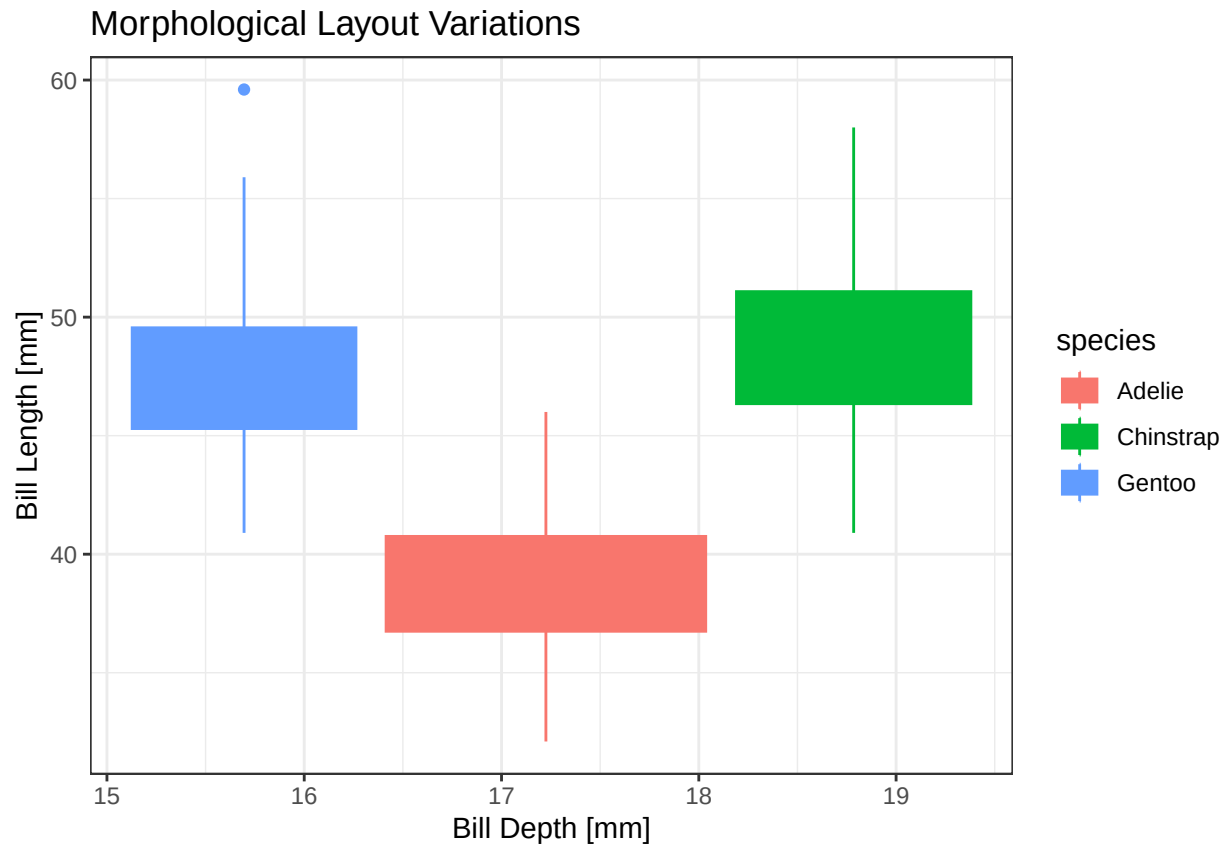
For instance, using a continuous line geometry (`geom_line()`) or a standard column layout (`geom_col()`) on non-sequential, individual specimen records is analytically incorrect. A continuous line should only be used when the X-axis represents a truly continuous variable (such as time-series data or an ordered trend).

Always ensure your geometric representation aligns with the nature of your data attributes.

```
# Establish a reusable global base structure for categorical distribution checks
baza <- ggplot(data = penguins,
               aes(x = bill_depth_mm,
                   y = bill_length_mm,
                   col = species,
                   fill = species)) +
  labs(title = "Morphological Layout Variations",
       x = "Bill Depth [mm]",
       y = "Bill Length [mm]")
```

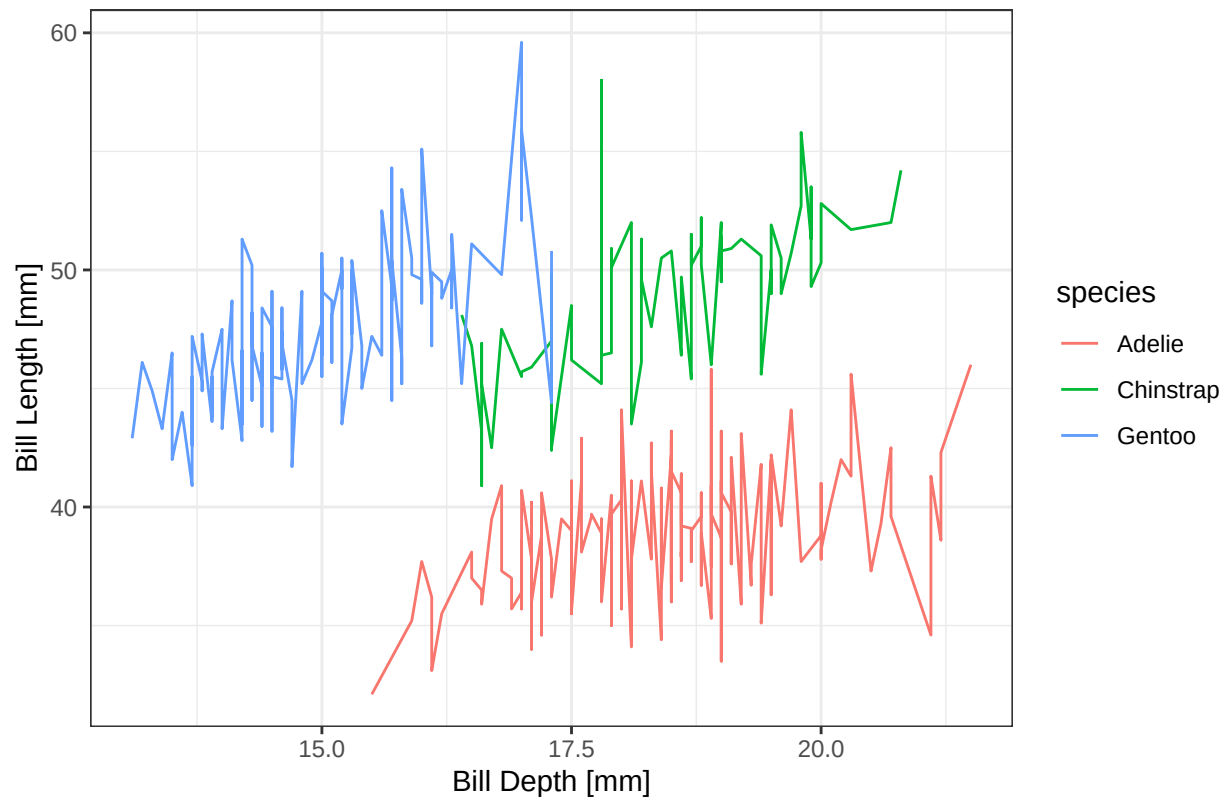
Comparing Geometric Output Formats

```
# Standard Boxplot representation of metrics  
baza + geom_boxplot()
```



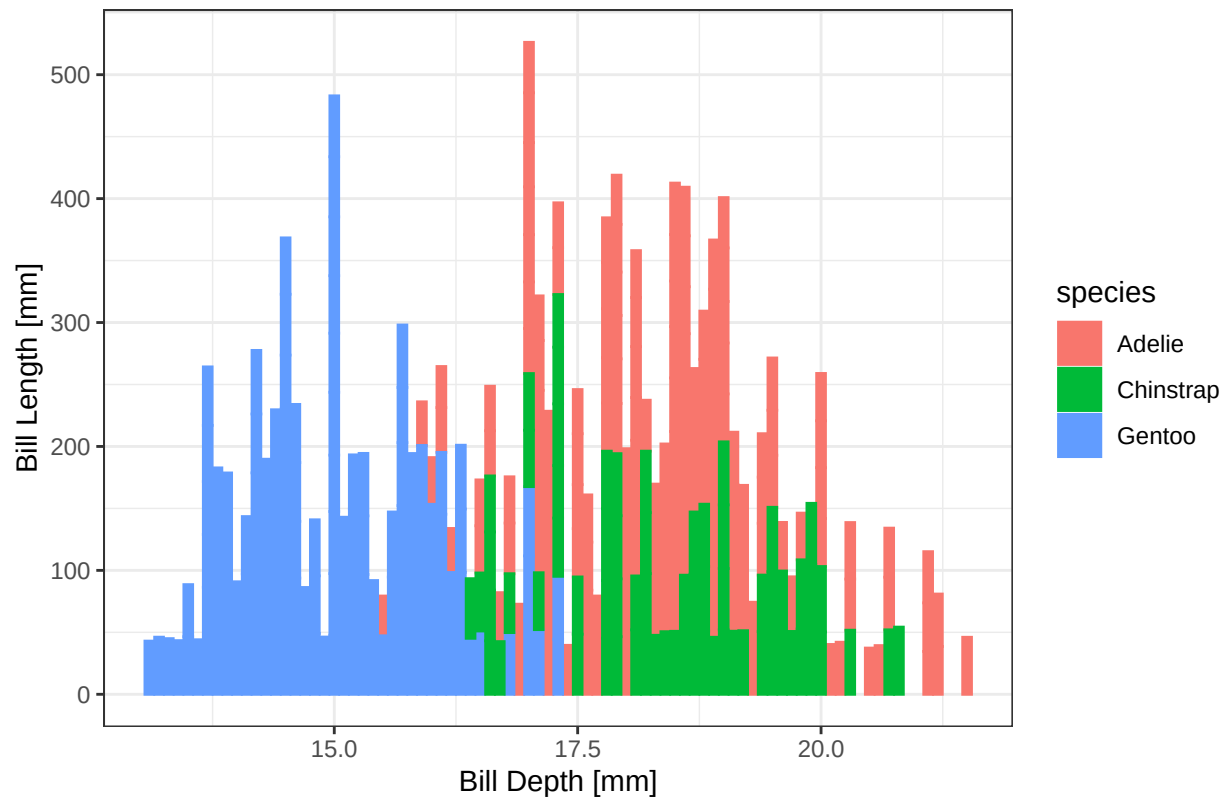
```
# Continuous Line representation  
baza + geom_line()
```

Morphological Layout Variations

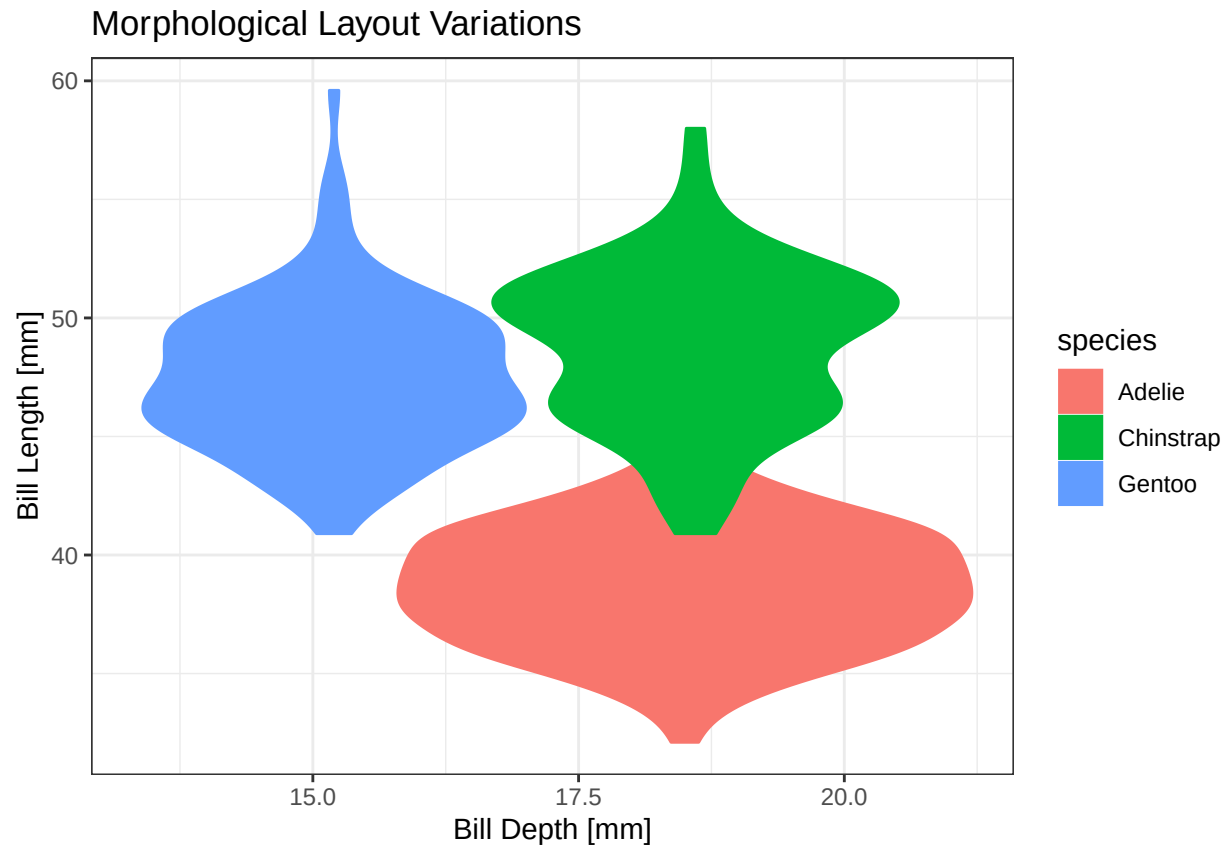


```
# Geometric Column layout  
baza + geom_col()
```

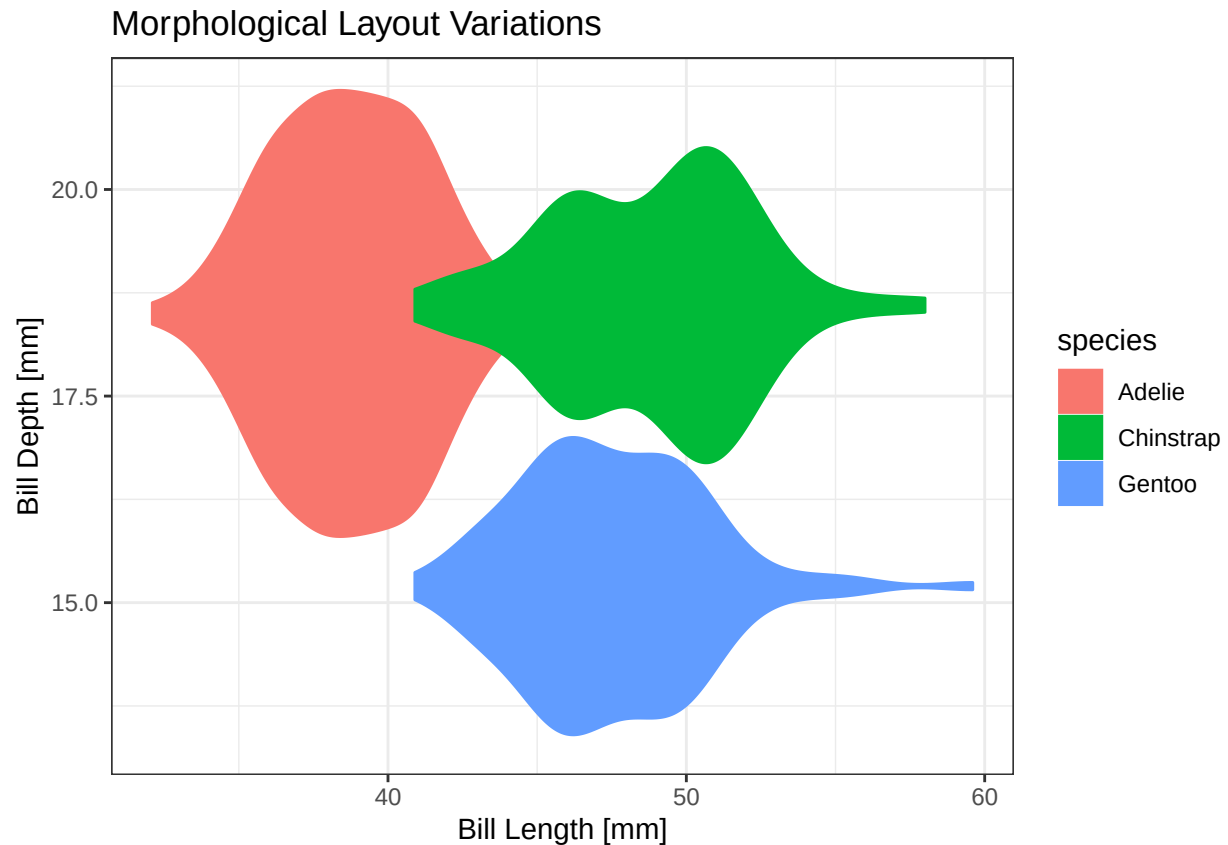
Morphological Layout Variations



```
# Violin plot tracking kernel density probability distributions  
baza + geom_violin()
```



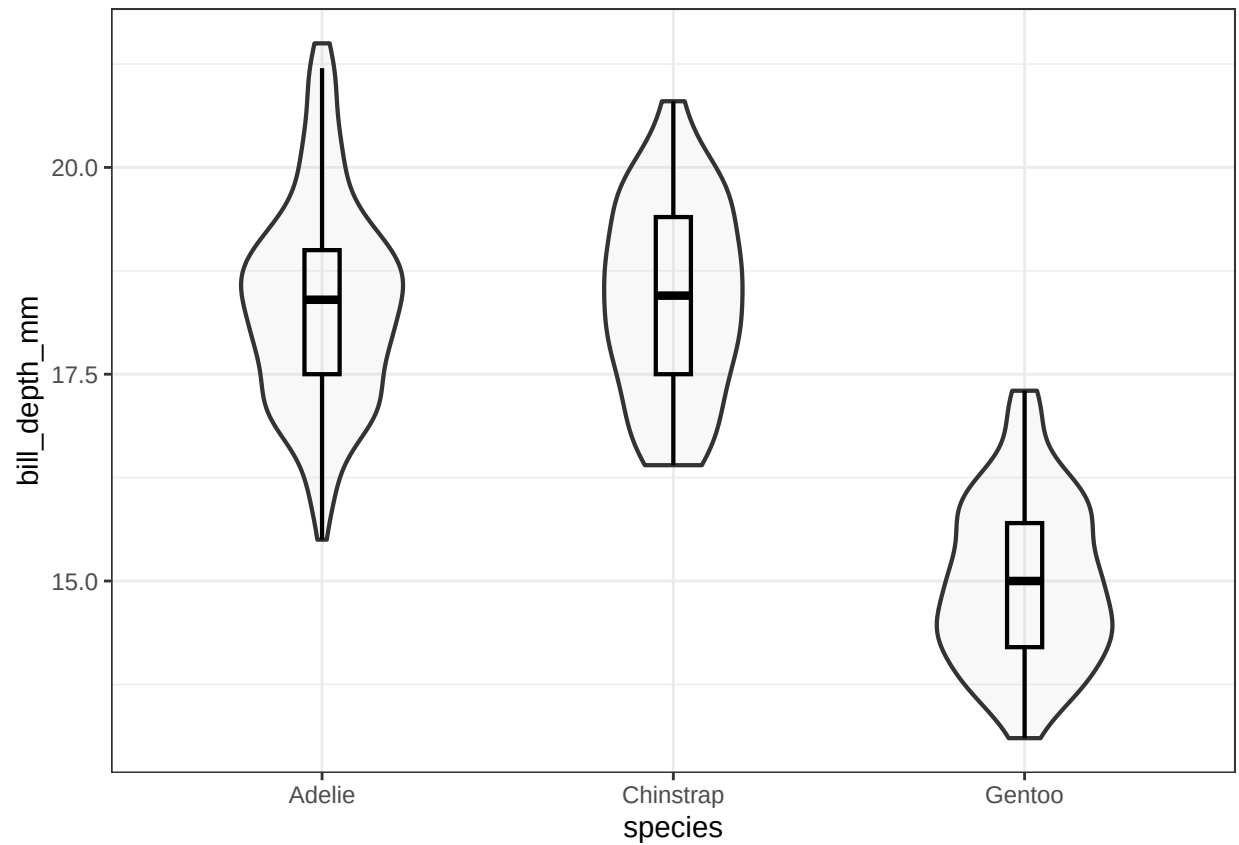
```
# Inverting default Cartesian layout grids using coordinate flips  
baza + geom_violin() + coord_flip()
```



Hybrid Geometry Composition (Layer Stacking)

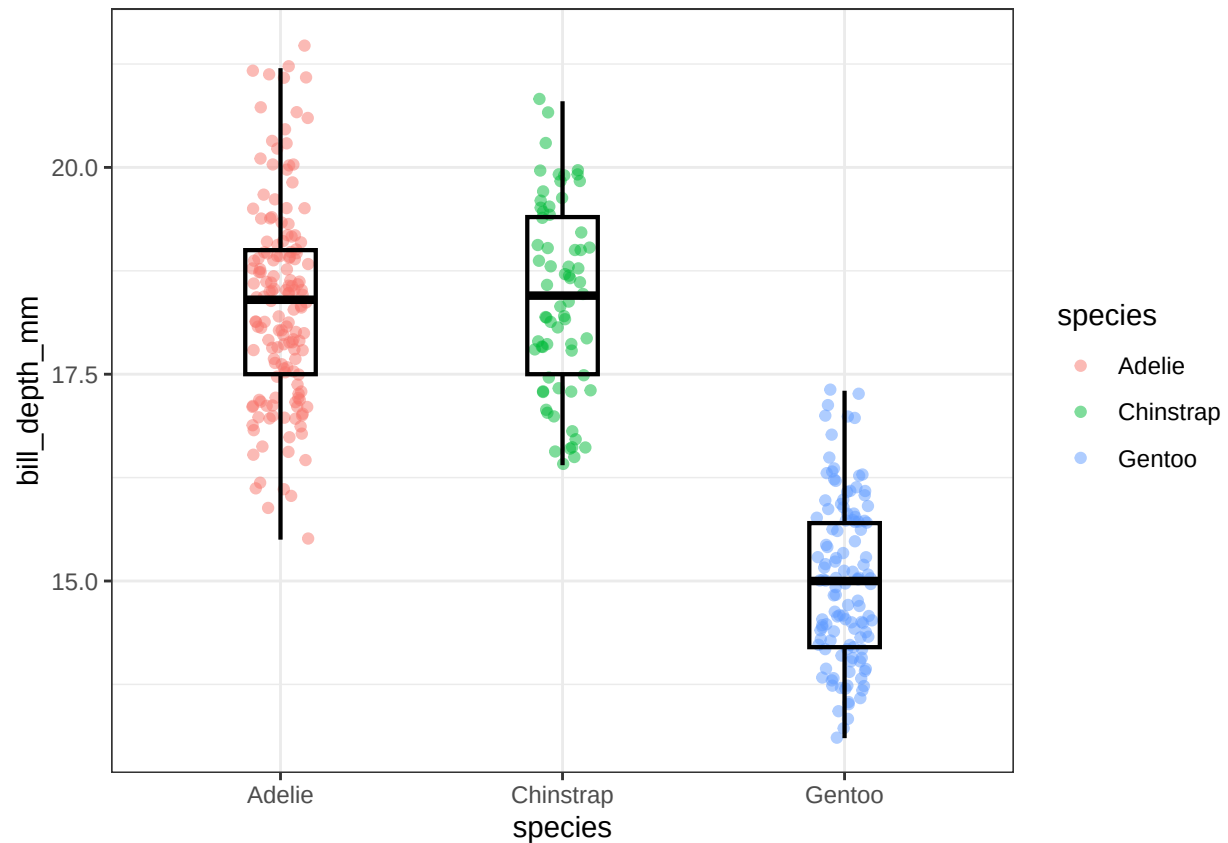
We can stack multiple geometries to maximize data transparency. For example, overlaying a boxplot on top of a semi-transparent violin plot provides a comprehensive view of sample density and summary statistics.

```
# Composite box-whisker and violin density layout
ggplot(penguins, aes(x = species, y = bill_depth_mm)) +
  theme_bw() +
  geom_violin(alpha = 0.1, size = 0.75, width = 0.5, fill = "gray") +
  geom_boxplot(alpha = 0, size = 0.75, width = 0.1, color = "black")
```



Intermediate Composite Plotting with Custom Jitter Overlays

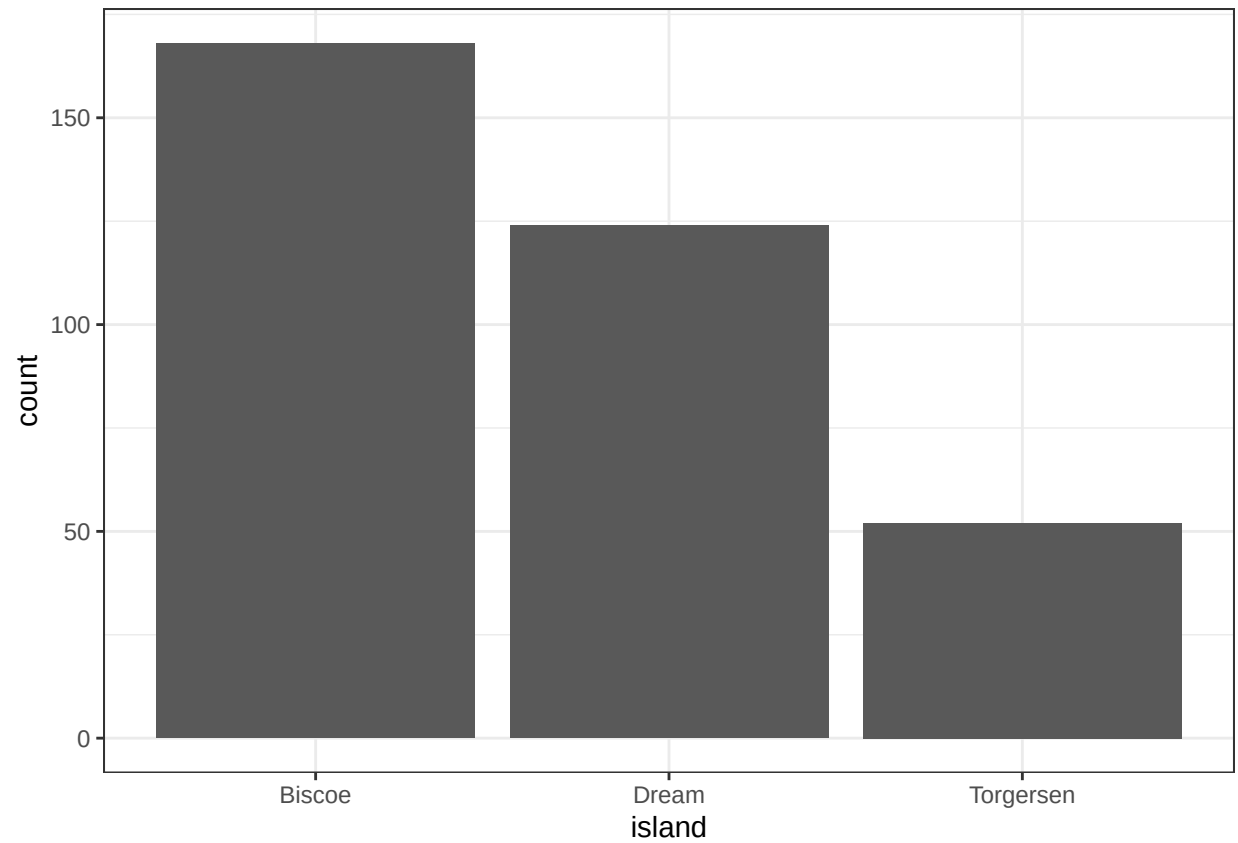
```
# Colored scatter observations with matching structural boxplots stacked on top
ggplot(penguins, aes(x = species, y = bill_depth_mm, col = species)) +
  theme_bw() +
  geom_point(alpha = 0.5, position = position_jitter(width = 0.1)) +
  geom_boxplot(alpha = 0, size = 0.75, width = 0.25, color = "black")
```



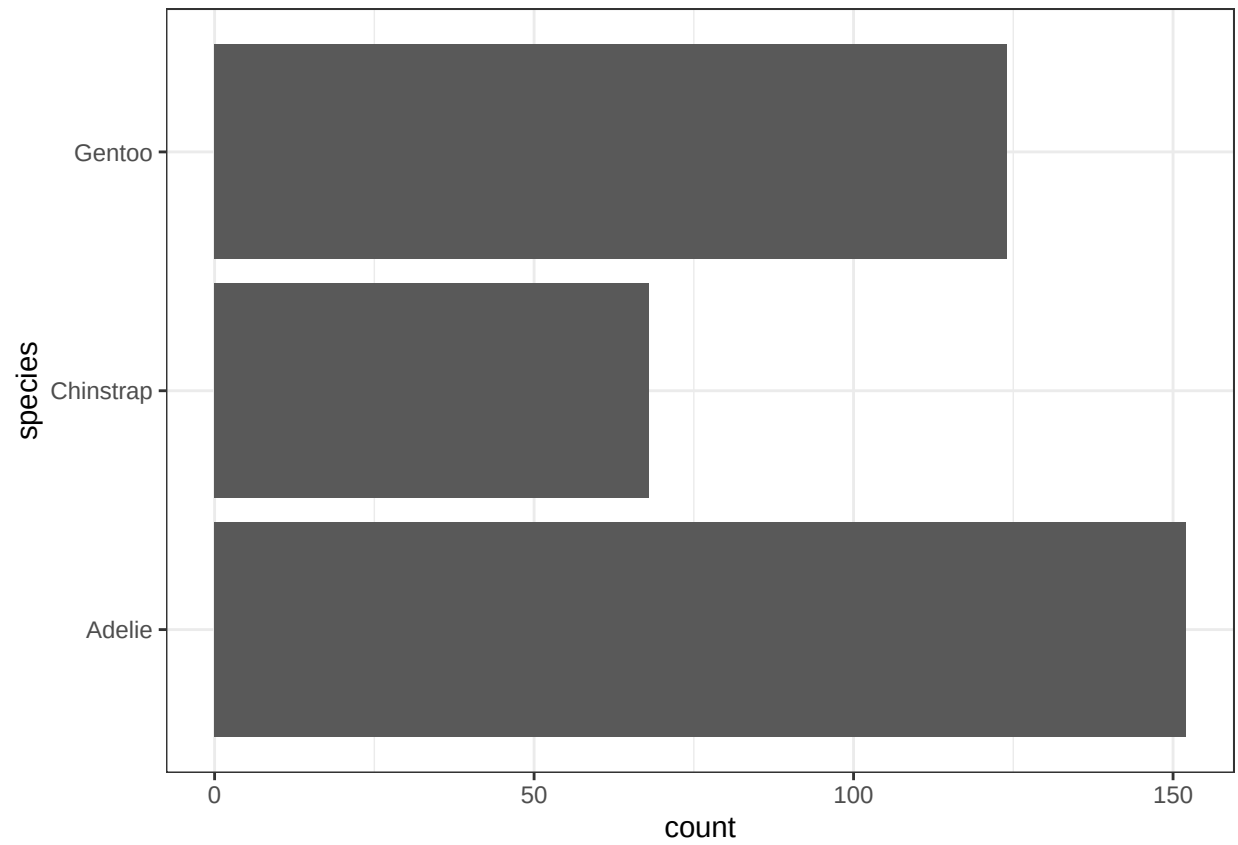
4. One-Dimensional Layouts: Categorical Factor Sorting

By default, coordinate systems plot categorical elements alphabetically. For professional charts, we should arrange categorical distributions by frequency using the `forcats` package.

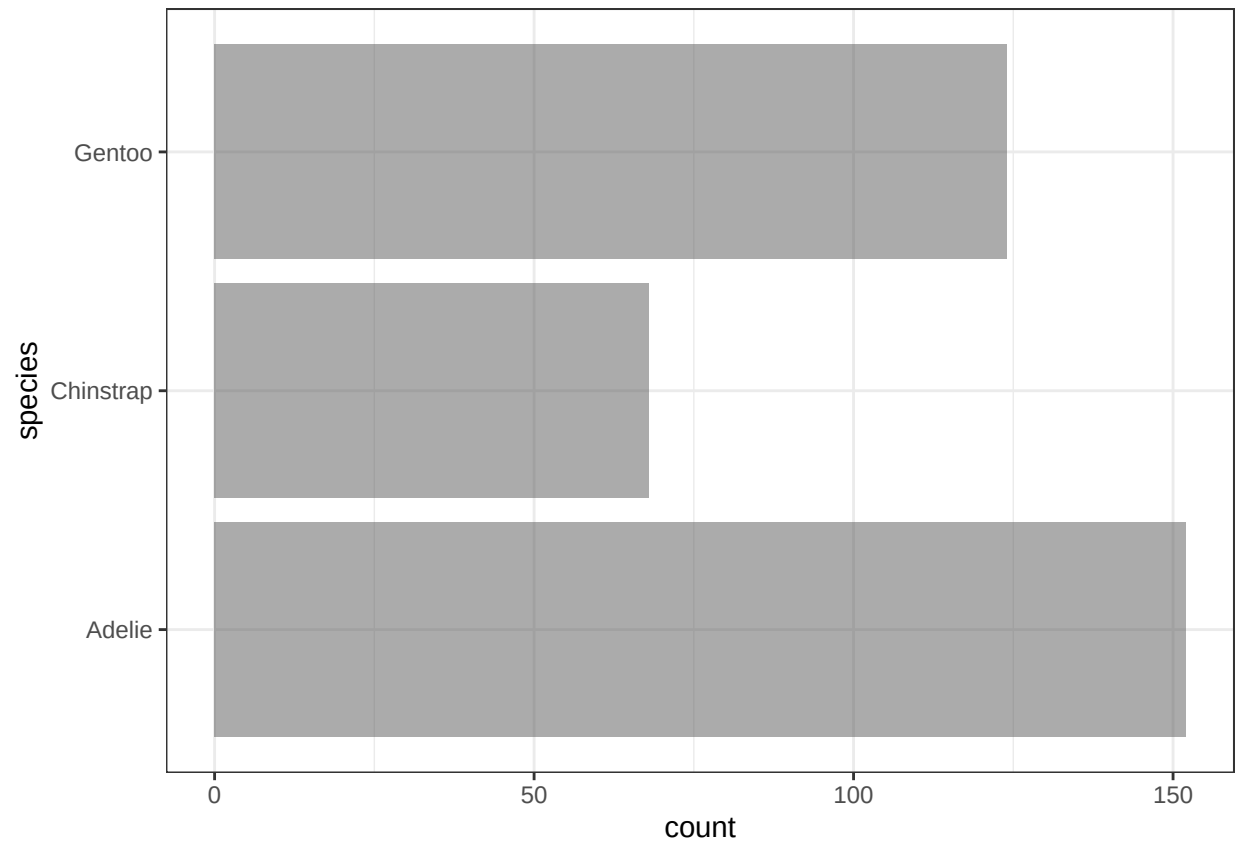
```
# Standard barplot calculating factor item counts automatically  
ggplot(penguins, aes(x = island)) +  
  geom_bar()
```

```
# Horizontal factor counts layout  
ggplot(penguins, aes(y = species)) +  
  geom_bar()
```

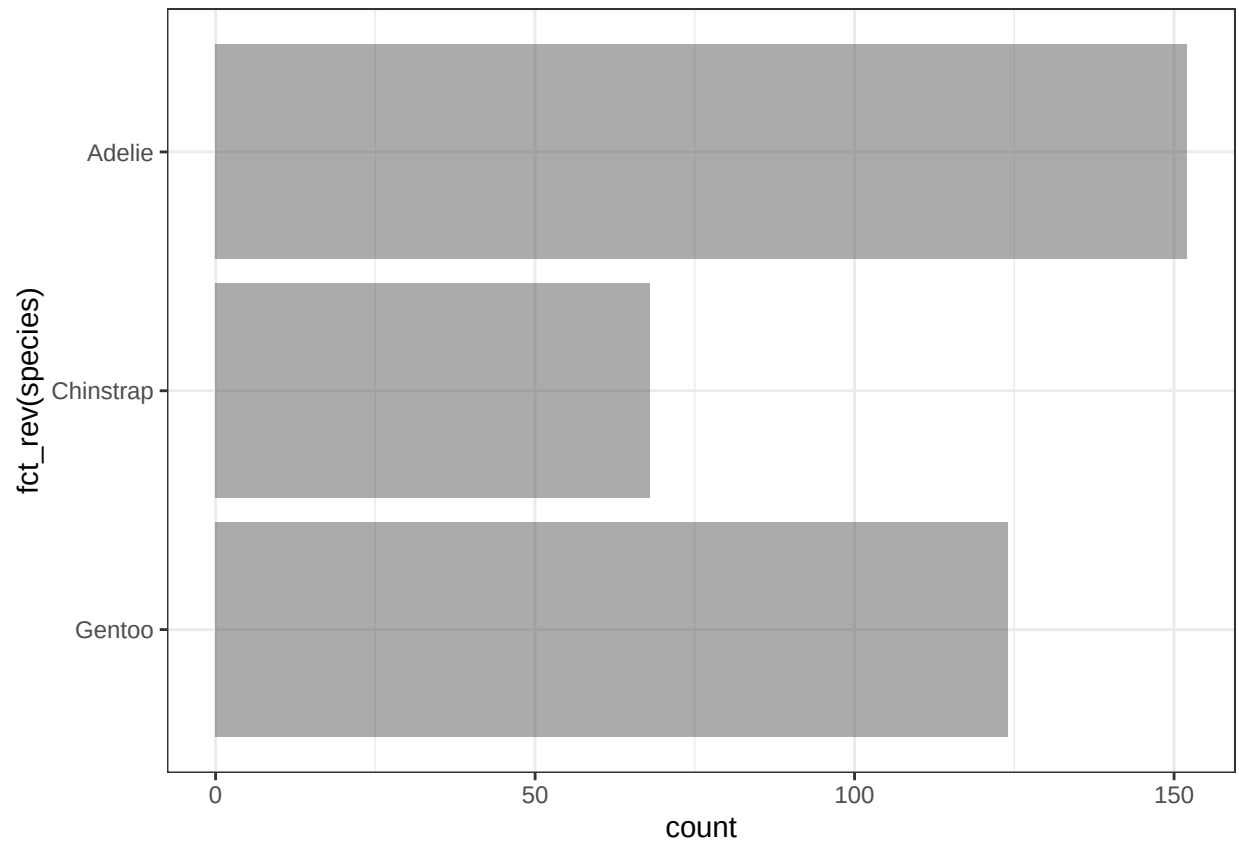


```
# Adjusting visual transparency parameters globally  
ggplot(penguins, aes(y = species)) +  
  geom_bar(alpha = 0.5)
```

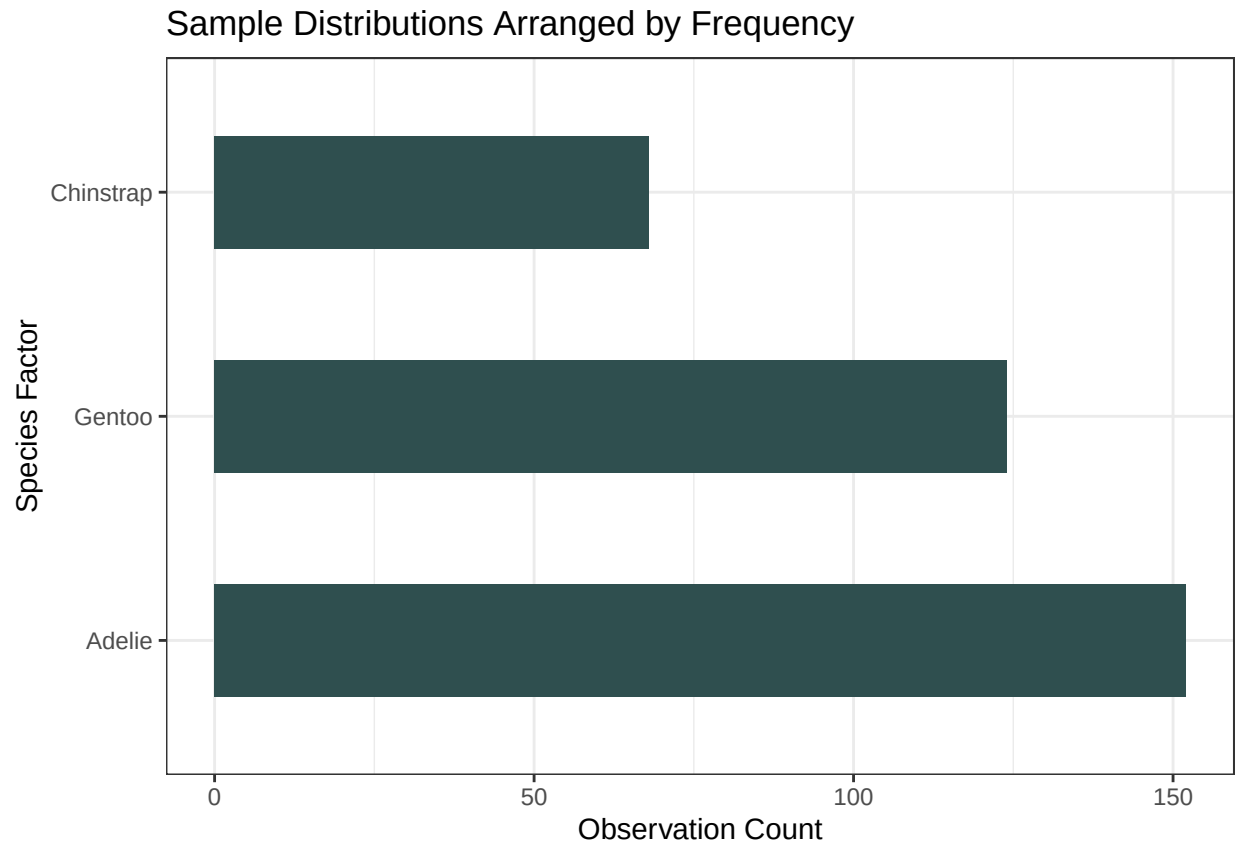


Factor Reordering Techniques

```
# Inverting alphabetical configurations with fct_rev()  
ggplot(penguins, aes(y = fct_rev(species))) +  
  geom_bar(alpha = 0.5)
```



```
# Reordering category factors dynamically by sample frequency with fct_infreq()
ggplot(penguins, aes(y = fct_infreq(species))) +
  geom_bar(width = 0.5, fill = "darkslategray") +
  labs(title = "Sample Distributions Arranged by Frequency",
       x = "Observation Count",
       y = "Species Factor")
```



5. Advanced Panel Layouts: Faceting and Marginal Displays

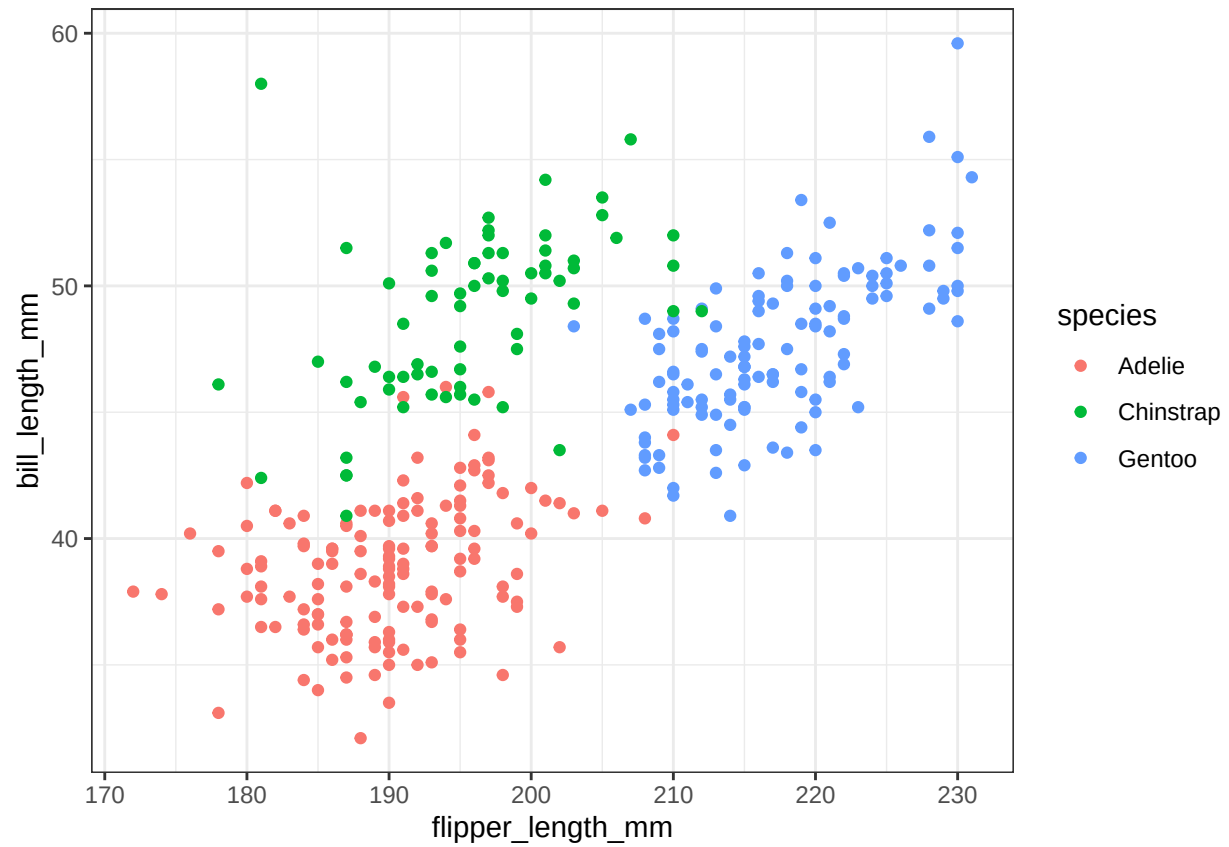
When single coordinates become cluttered, we can split plots into a matrix of smaller sub-panels using faceting, or append marginal plots to show individual distributions.

Split Axis Profiling via Matrix Facets

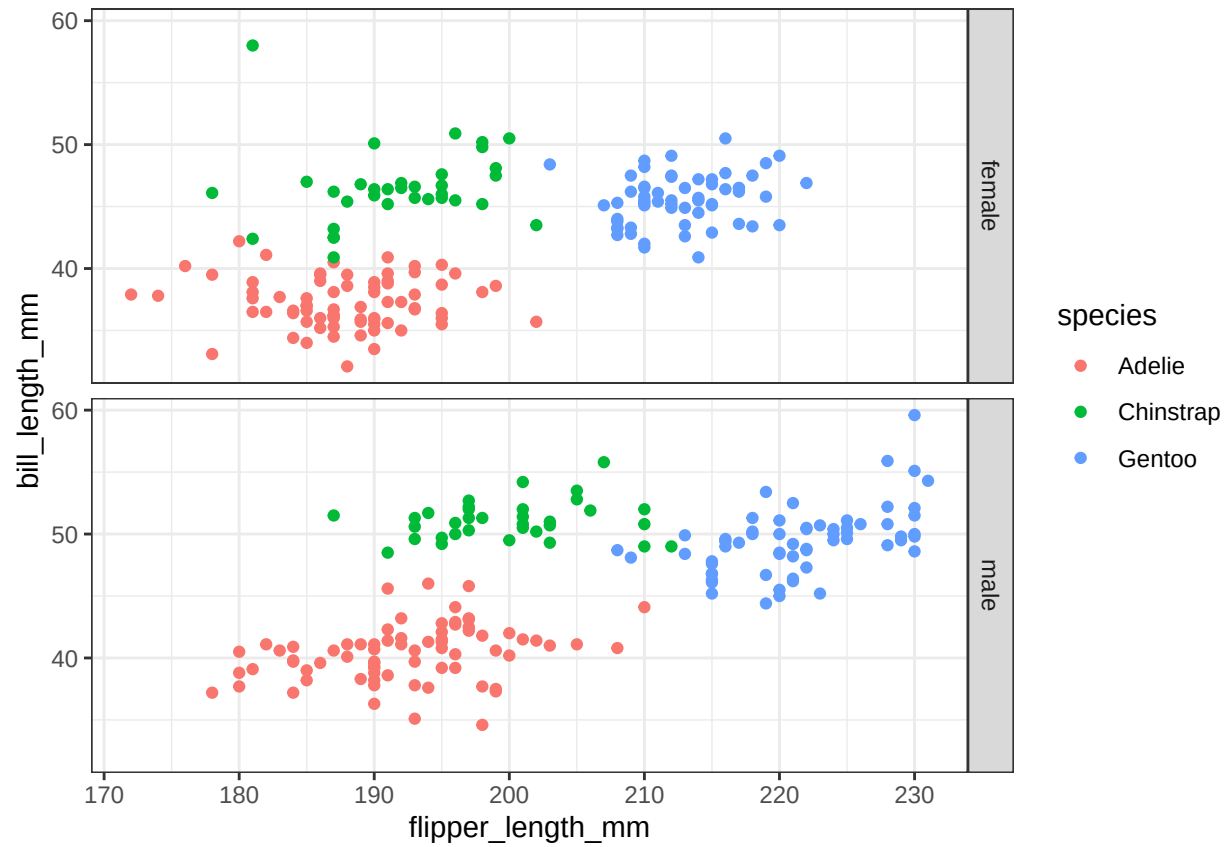
```
# Clean missing values to ensure clear facet panels
penguins_clean <- na.omit(penguins)

# Define our base scatter plot configuration
fbase <- ggplot(penguins_clean, aes(x = flipper_length_mm, y = bill_length_mm)) +
  geom_point(aes(color = species))

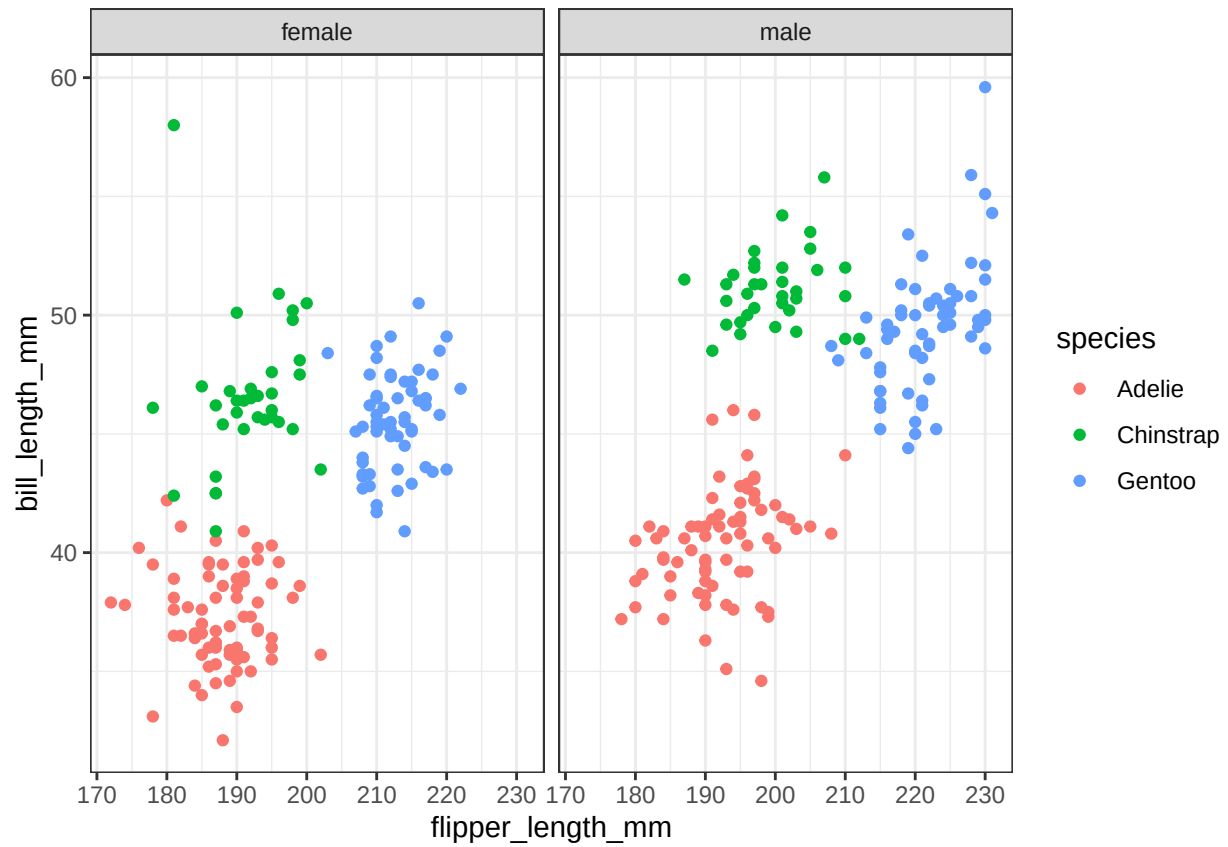
fbase
```



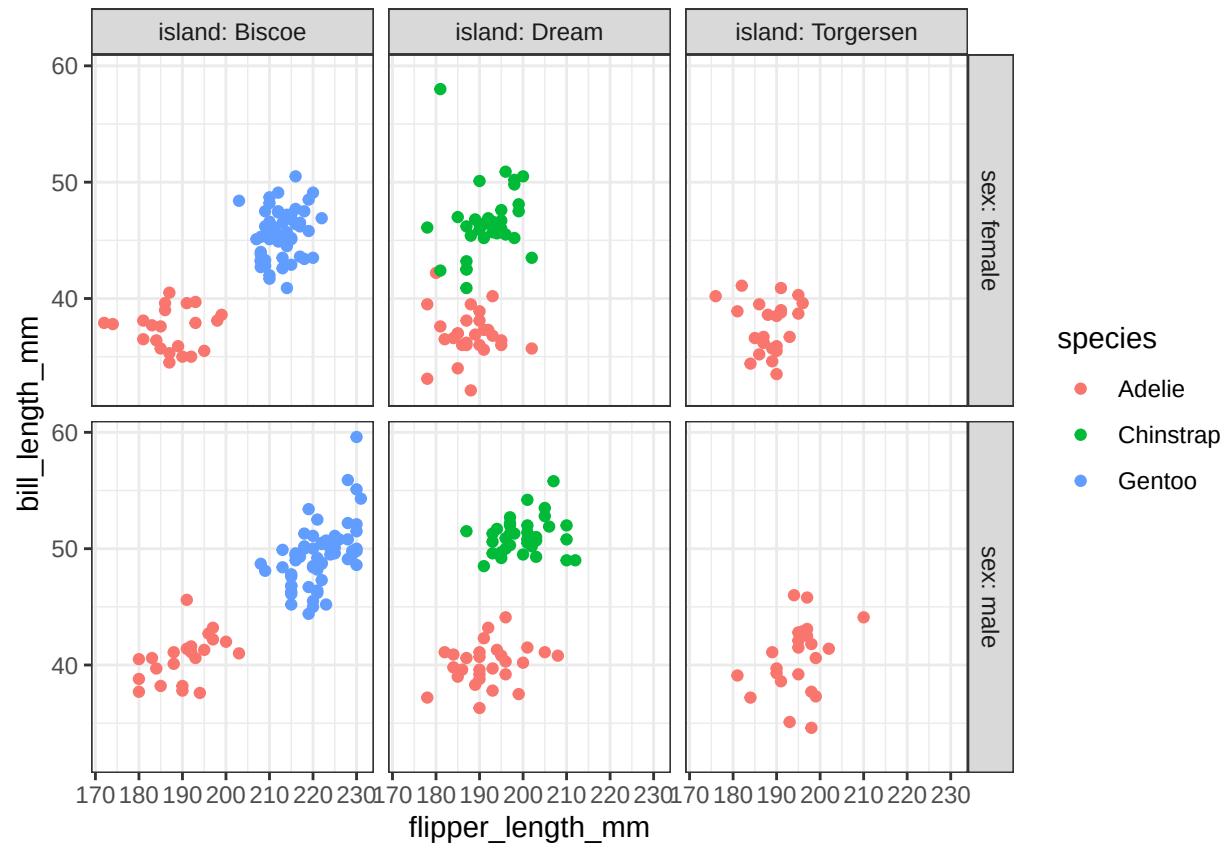
```
# Split into horizontal panel arrays segmented by sex
fbase + facet_grid(sex ~ .)
```



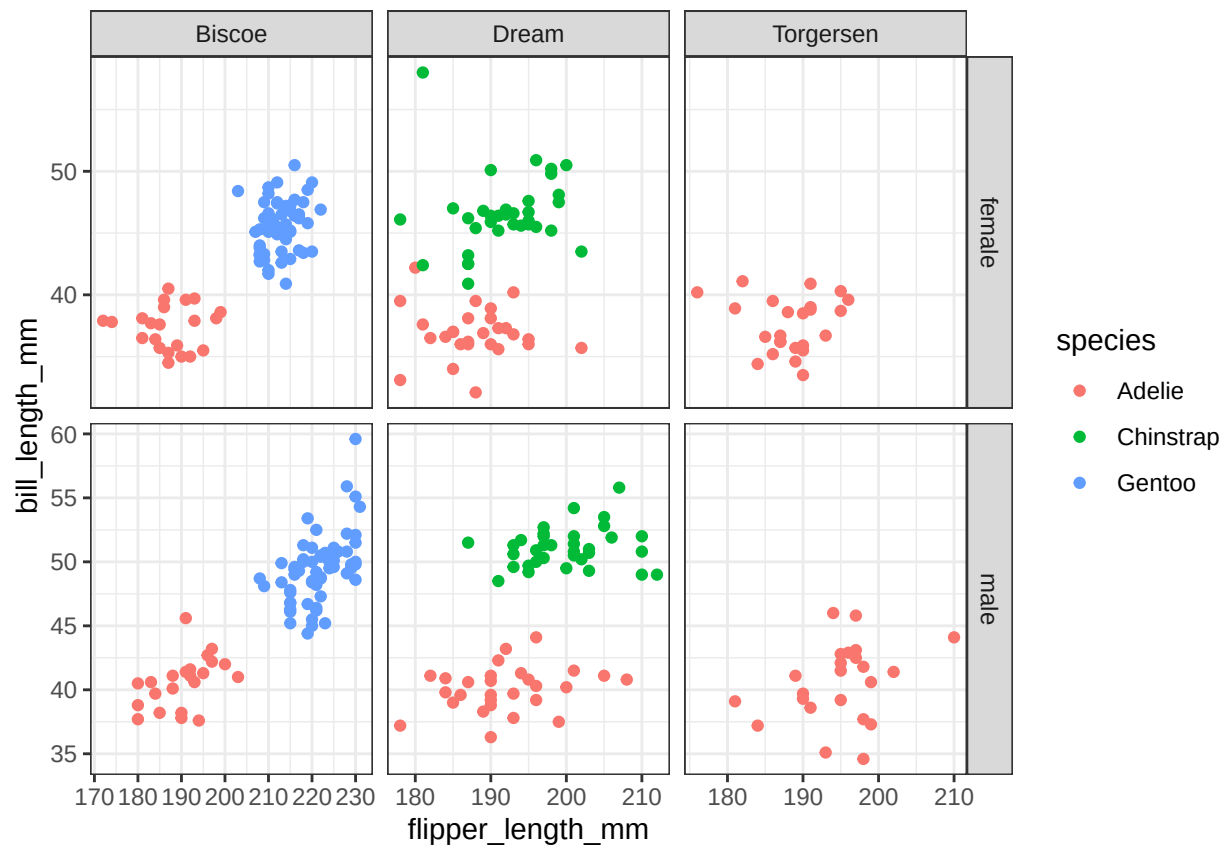
```
# Split into vertical panel arrays segmented by sex
fbase + facet_grid(. ~ sex)
```



```
# Multi-variable grid layout (Rows = Sex, Columns = Island) with dynamic labels
fbase + facet_grid(sex ~ island, labeller = label_both)
```

```
# Allow panel axis scales to vary independently across groups
fbase + facet_grid(sex ~ island, scales = "free")
```

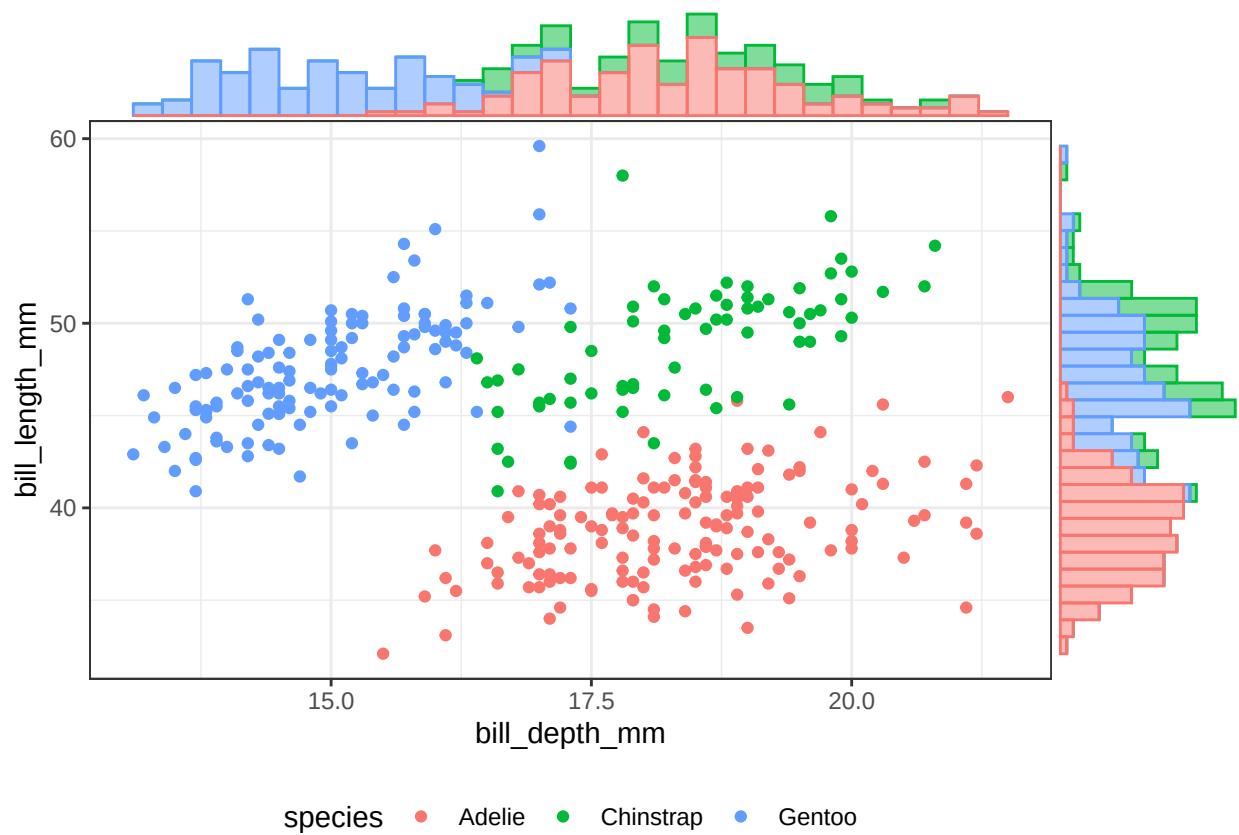


Appending Marginal Coordinates via ggExtra

To display individual distributions alongside a joint scatter plot, save the base scatter plot configuration and process it using `ggMarginal()`.

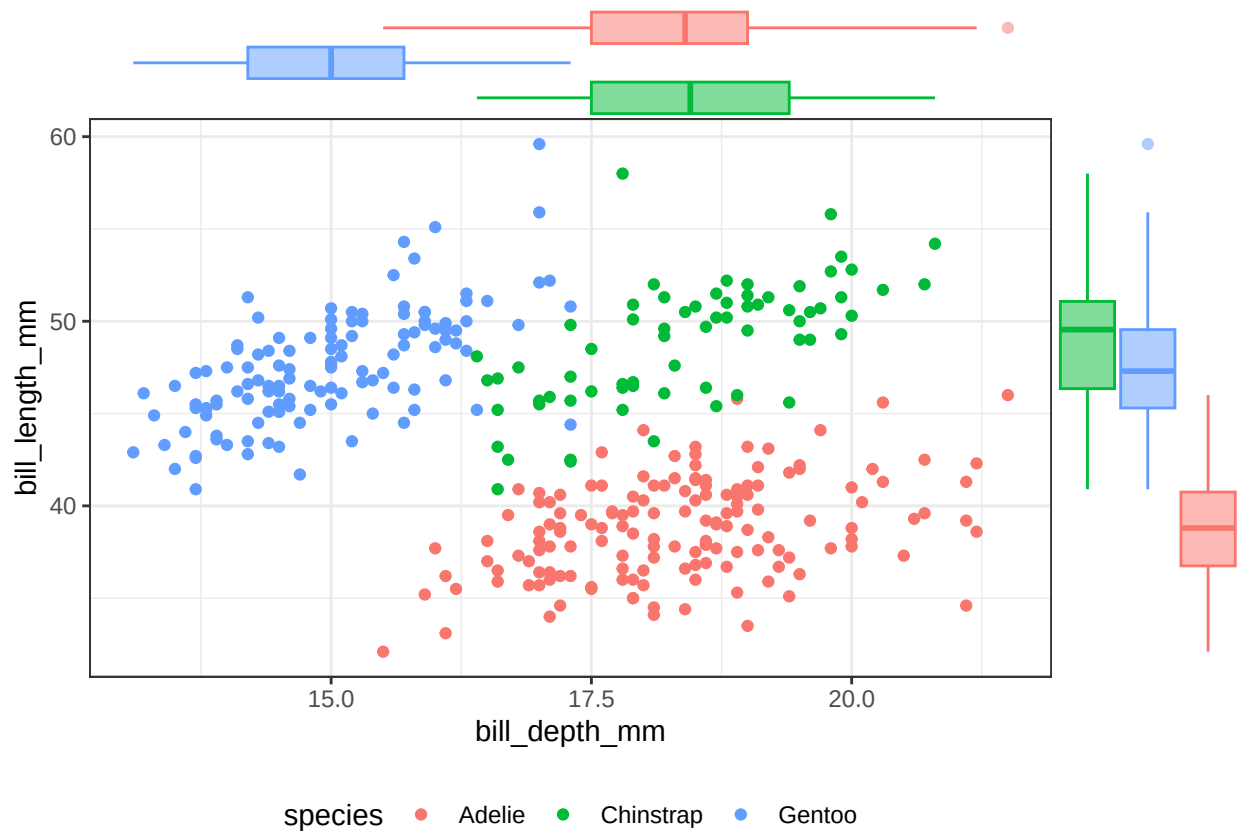
```
# Save base scatter configuration object
p <- ggplot(penguins, aes(x = bill_depth_mm, y = bill_length_mm, color = species)) +
  geom_point() +
  theme(legend.position = "bottom")

# Generate marginal histogram sub-elements
ggMarginal(p,
  type = "histogram",
  groupColour = TRUE,
  groupFill = TRUE)
```



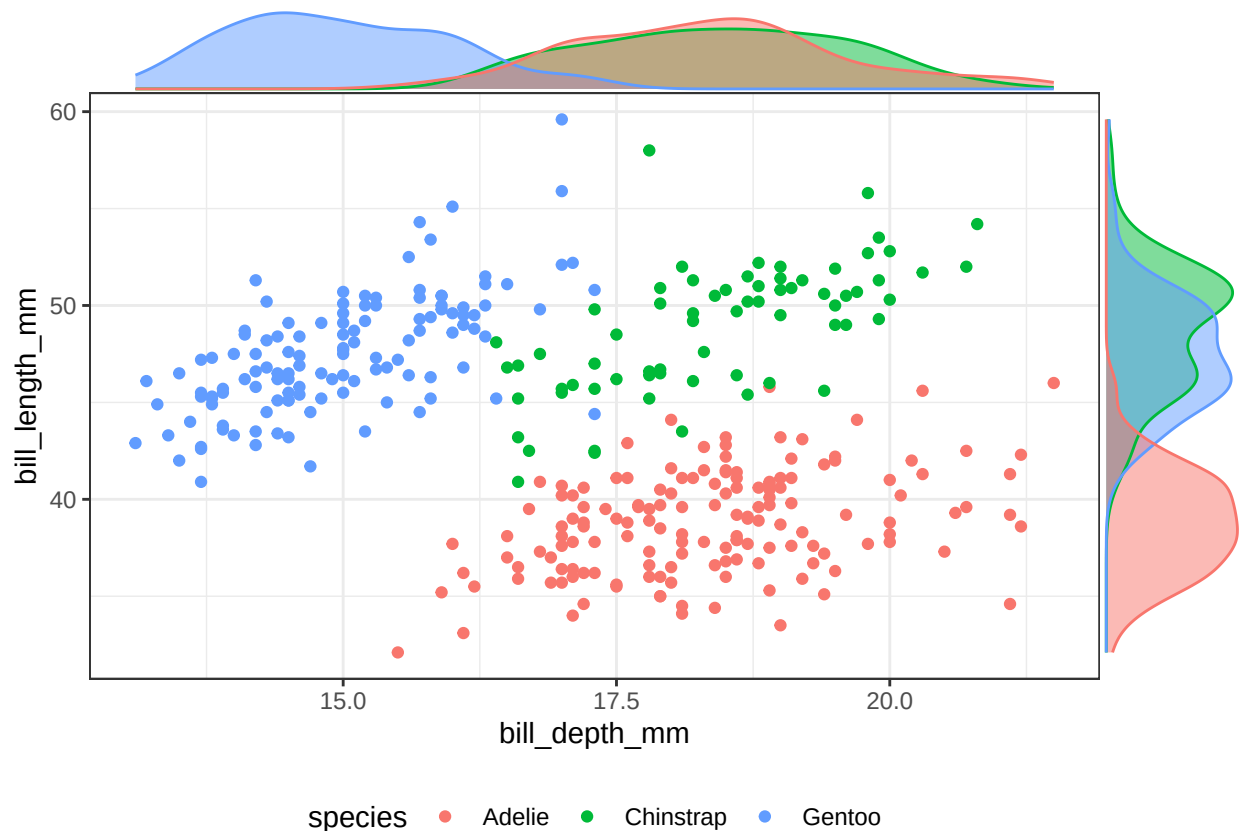
And with boxplot

```
# Generate marginal boxplot sub-elements
ggMarginal(p,
  type = "boxplot",
  groupColour = TRUE,
  groupFill = TRUE)
```



This time density plot:

```
# Plot 3: Scatter with elegant, narrow marginal DENSITY plots
ggMarginal(p,
  type = "density",
  groupColour = TRUE,
  groupFill = TRUE,
  size = 7) # Increases the main plot area, making marginal plots narrower
```



Things I Found Useful & Interesting

DataViz Theory & Visual Communication Best Practices

- Four Steps to Better Data Visualizations – Practical framework for creating more impactful visualizations.
- Simplify charts (Medium) – An insightful read on why simplicity remains the ultimate goal of chart design.
- Storytelling & Design Chart (Medium) – Straightforward design techniques to capture and retain reader attention.
- Edward Tufte: Chartjunk – A classic introduction to identifying and eliminating visual clutter from charts.
- Autodesk: Datasaurus Dozen – A brilliant demonstration of why visualizing data is critical (shows datasets with identical summary statistics but completely different distributions).
- The Data Visualisation Catalogue – A comprehensive reference library for choosing chart types.

The Grammar of Graphics & ggplot2 Documentation

- Hadley Wickham: A Layered Grammar of Graphics (PDF) – The foundational academic paper detailing the structural framework of `ggplot2`.
- Hadley Wickham: `ggplot2`. Elegant Graphics for Data Analysis (3rd Ed, online) – The complete, free online edition of the official handbook.
- Claus O. Wilke: Fundamentals of Data Visualization – An exceptional online textbook focusing on the principles of accurate data presentation.

- [STHDA: Be Awesome in ggplot2 \(Practical Guide\)](#) – Excellent practical reference guide for building highly polished charts.
- [Datanovia: GGplot Examples Best Reference](#) – Superb reference source covering standard plots and extension packages.
- [Top 50 Ggplot2 Visualizations Master List](#) – A repository of the 50 most common chart types with reproducible R code.
- [The R Graph Gallery: ggplot2 section](#) – An interactive showcase of inspiration and ready-to-use templates.